COSC264 · INTRODUCTION TO COMPUTER NETWORKS AND THE INTERNET · TERM 4

# Module 1
# Introduction to Routing

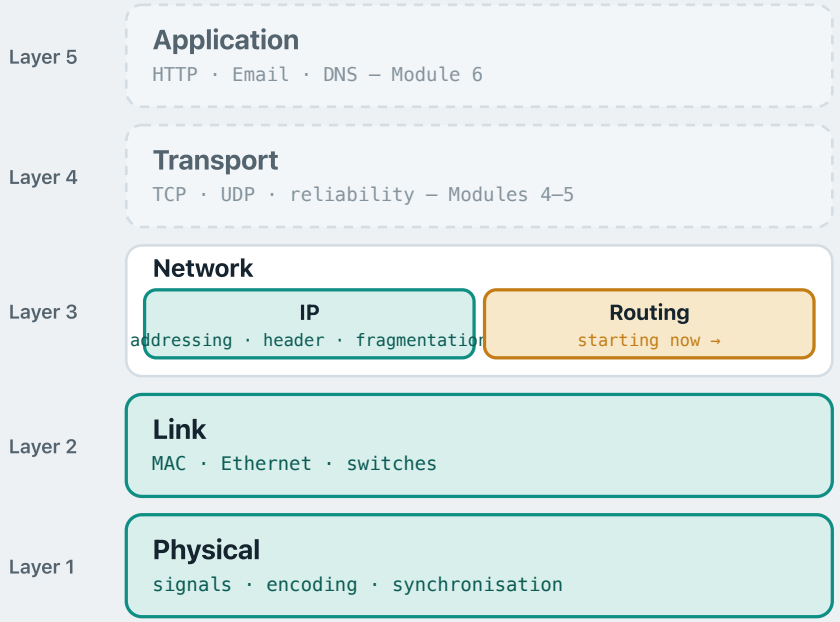
1 · The Routing Problem | 2 · Shortest Paths & Distance-Vector | 3 · Dijkstra & Link-State Routing

University of Canterbury · Yinuo Zhang · yinuo.zhang@canterbury.ac.nz

cosc264.up.railway.app

RECAP AND OVERVIEW

# An overview of the rest of the course




THE FIRST HALF OF THE COURSE

- The Internet stack is divided into **five layers**.
- We covered layers **1** and **2** — physical and link — and half of layer **3**.

FROM HERE ON

- The missing half of layer **3** is **routing** — that is what we start on now.
- The two layers above — **transport** and **application** — follow in the rest of the course.

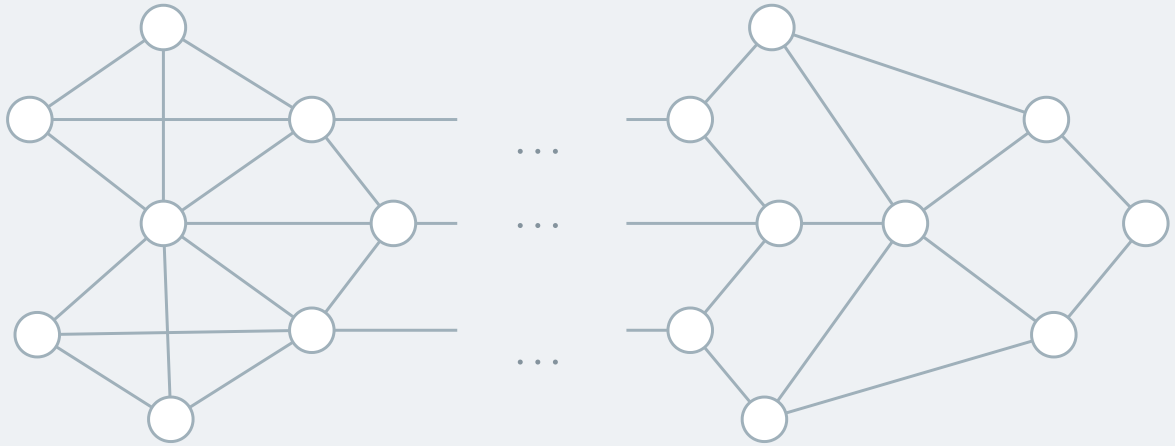
MODULE 1

# The Routing Problem

1 · The Routing Problem | 2 · Shortest Paths & Distance–Vector | 3 · Dijkstra & Link–State Routing

QUESTION 1 OF 4

# Why do we need routing?



Suppose we have a large network. The diagram above shows part of it — there are many more routers and links in the middle.

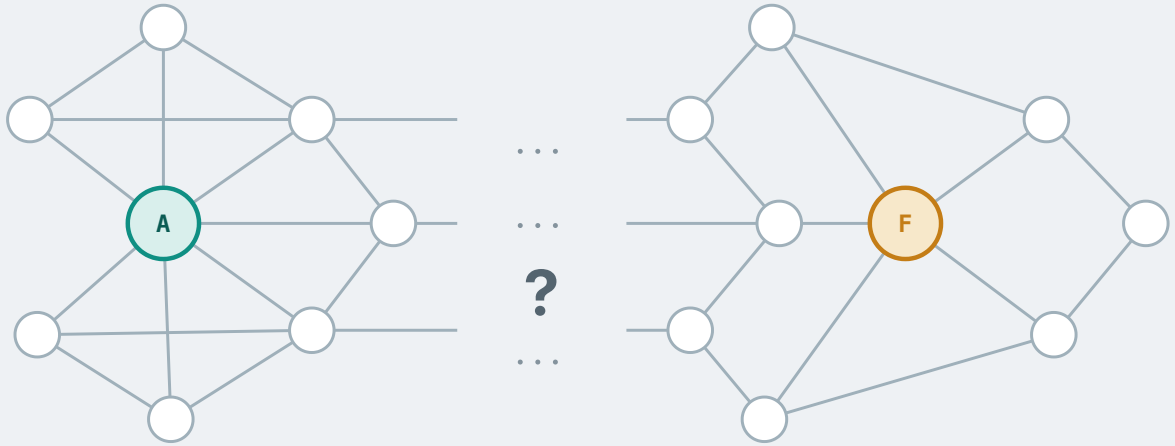
QUESTION 1 OF 4

# Why do we need routing?


DEFINITION A **path** from A to F is a sequence of routers $R_0$, $R_1$, …, $R_k$ with $R_0$ = A and $R_k$ = F, where every consecutive pair is joined by a link. The message is handed along the whole sequence.

QUESTION 1 OF 4

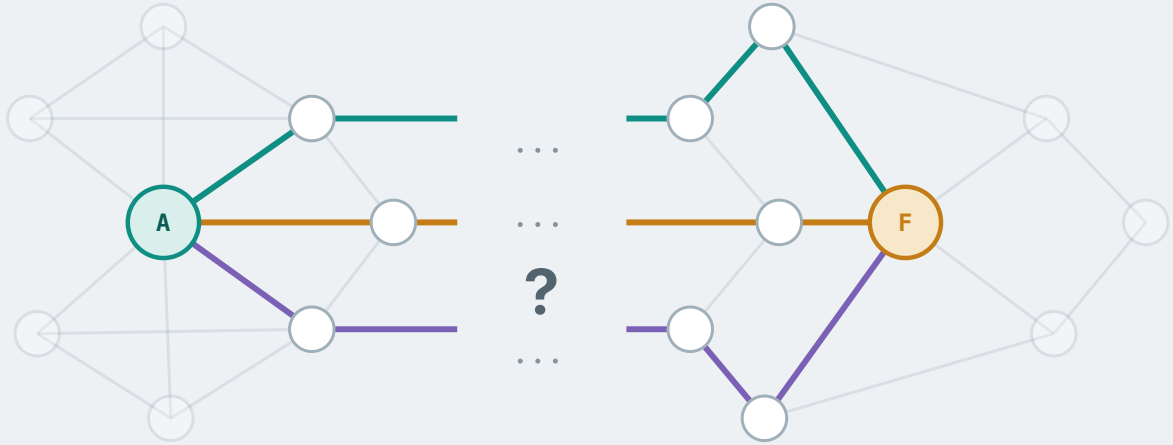
# Why do we need routing?




How does the network know **which path to use** for this source and this destination? **routing!**

QUESTION 1 OF 4

# Why do we need routing?




Furthermore, it is often the case that **more than one path exists**. When several would do, how does the network know **which is the best one** to use? **routing!**

QUESTION 2 OF 4

# What does routing do?

Routing is how a network decides, **automatically**, which path packets take from each source to each destination.

`which path counts as best`

Routing does not hand out just *any* path. It defines a **cost metric**, so that every path has a cost — and then picks the path with the **least** cost.

The cost metric can capture many factors, such as hop count (distance), delay, bandwidth, and so on.

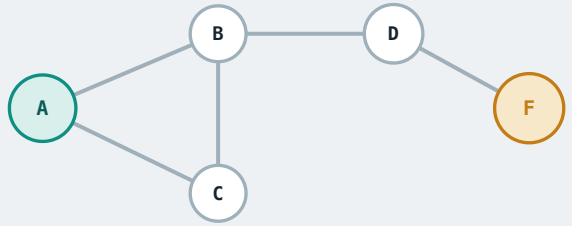
**`when it happens`**

- **Before any packet arrives.** Working a path out per message, on the fly, would be far too slow — the answer has to be ready and waiting by the time a message turns up.
- **And again whenever the network changes.** Links fail and costs change, so a path that was best a moment ago may not even exist now. Routing is therefore always running.

WHAT ROUTING PRODUCES

# How is the path information stored? — the forwarding table

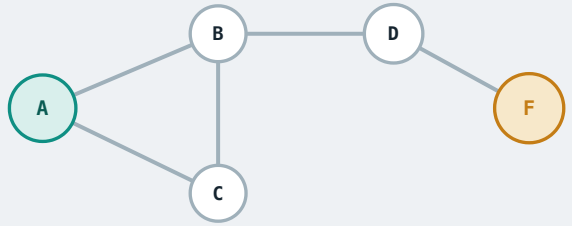
Routing works out a path from **every source to every destination**. But where does that information actually **live** once it has been worked out?



WHAT ROUTING PRODUCES

# How is the path information stored? — the forwarding table

In a **forwarding table**, and every router keeps one. It records, for each destination, which **next hop** to send it to — and nothing more.



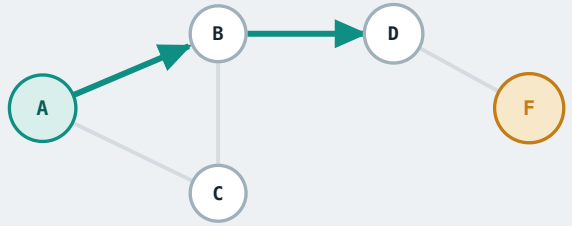

A's table

| DEST | NEXT HOP |
|---|---|
| B | B |
| C | C |
| D | B |
| **F** | **B** |

WHAT ROUTING PRODUCES

# How is the path information stored? — the forwarding table

B now does exactly the same thing with *its own* table: for destination **F**, next hop is **D**. So B hands it to D.




A’s table

| DEST | NEXT HOP |
|---|---|
| B | B |
| C | C |
| D | B |
| **F** | **B** |

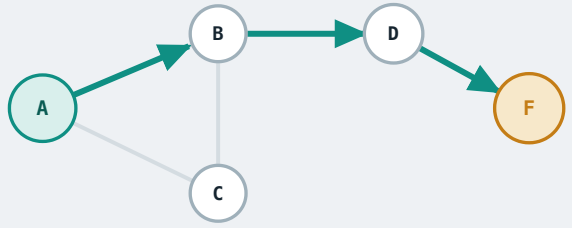
B’s table

| DEST | NEXT HOP |
|---|---|
| A | A |
| C | C |
| D | D |
| **F** | **D** |

WHAT ROUTING PRODUCES

# How is the path information stored? — the forwarding table

Notice that each router holds only a **local** decision — none of them has the global path. Put those local decisions together and you get **A → B → D → F**.




**A → B → D → F**

A's table

| DEST | NEXT HOP |
|---|---|
| B | B |
| C | C |
| D | B |
| **F** | **B** |

B's table

| DEST | NEXT HOP |
|---|---|
| A | A |
| C | C |
| D | D |
| **F** | **D** |

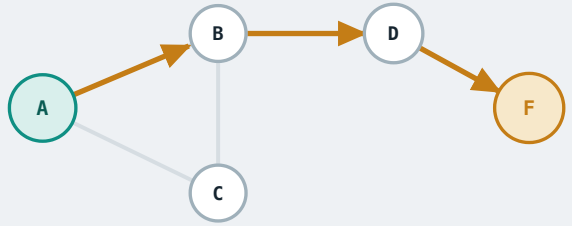
D's table

| DEST | NEXT HOP |
|---|---|
| A | B |
| B | B |
| C | B |
| **F** | **F** |

The goal of **routing**, therefore, is to build those **forwarding tables**.

Reading one row and passing the message on — what we just did — is **forwarding**.

# Routing vs. forwarding




A → B → D → F

A’s table

| DEST | NEXT HOP |
|---|---|
| B | B |
| C | C |
| D | B |
| F | B |

B’s table

| DEST | NEXT HOP |
|---|---|
| A | A |
| C | C |
| D | D |
| F | D |

D’s table

| DEST | NEXT HOP |
|---|---|
| A | B |
| B | B |
| C | B |
| F | F |

| | ROUTING | FORWARDING |
|---|---|---|
| job | produces the tables | uses the tables |
| plane | control plane | data plane |
| when | before data arrives | when a datagram arrives |
| speed | slower | faster — nanoseconds |

QUESTION 3 OF 4

# How do we do routing?

Easier to state than to do. Every router knows only the links it is plugged into — **none of them can see the whole network** — and yet a path has to be found across all of it.

`a routing algorithm`

Solves the routing problem under **ideal assumptions**.

**`a routing protocol`**

Embeds a routing algorithm into **a real networking context**:

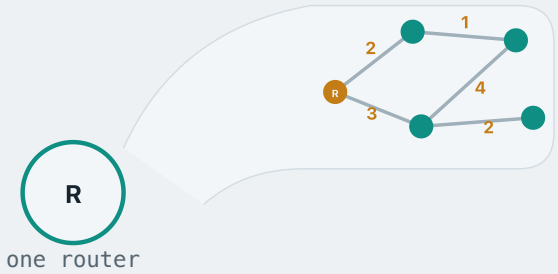
- it operates in a **distributed** environment;
- it carries that **information exchange** between the routers using **existing Internet protocols**;
- that exchange **takes time and can fail**, and the protocol has to allow for it.

# Two families of routing algorithms

**link-state**

distance-vector

# Link-state: get the whole map

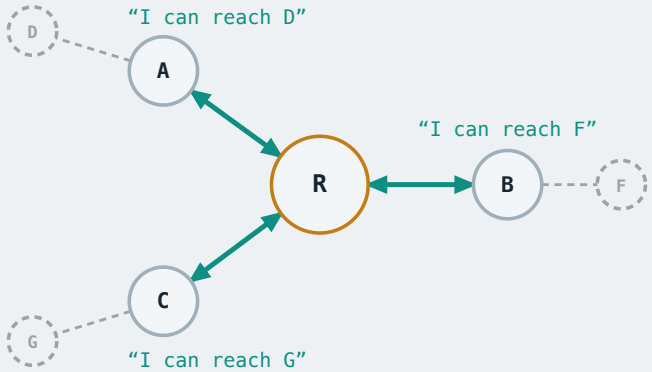



The routers talk to each of their **neighbours**.

They exchange information, so that at the end every router holds a **global view** of the network.

With that map in hand, each router works out its own forwarding table **locally** (next lecture).

# Distance-vector: ask your neighbours




**R's forwarding table**

| DEST | NEXT HOP |
|---|---|
| D | A |
| F | B |
| G | C |

The routers talk to each of their **neighbours** — exactly as before.

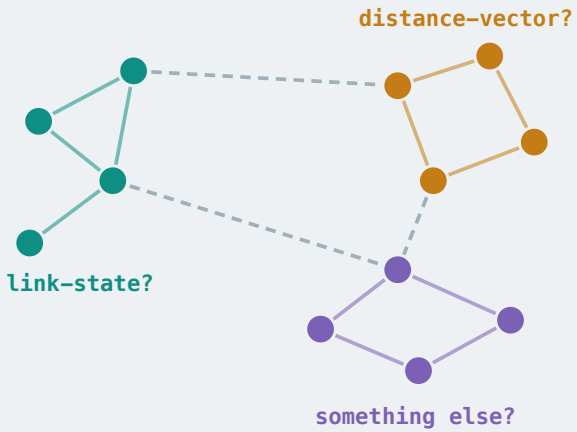
This time, instead of learning a global view, each router directly learns, for each destination, **which neighbour is the best next hop to forward to** for that destination.

Eventually each router builds its forwarding table out of that (next lecture).

QUESTION 4 OF 4

# Who runs the routing protocol?

A network is a great many routers — and they are not all run by the same people.




colour = the organisation that operates the router

One operator may want **link-state**. Another may prefer **distance-vector**. A third may not want to publish how its network is wired at all.

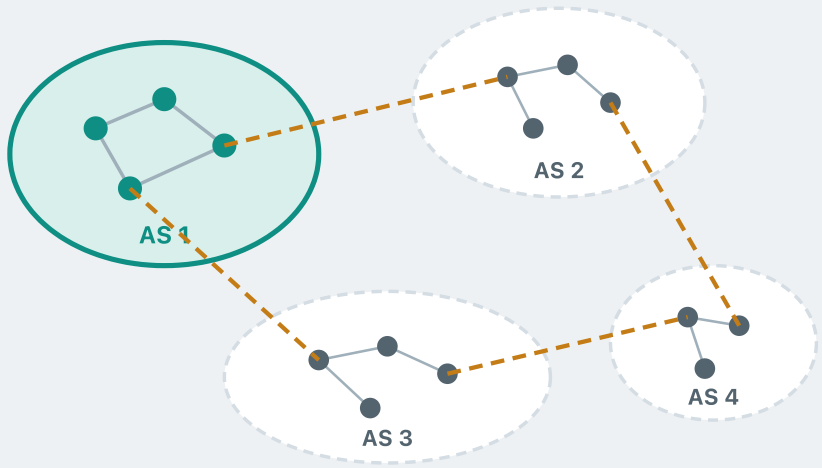
So **who decides** which routing protocol runs? Nobody is in charge of the Internet, so nobody gets to impose one answer.

The way out is to stop thinking of the Internet as a whole. It is a **network of subnetworks**.

So find the smallest piece that has the **autonomy** to decide for itself — the Internet is a network of **those**.

AUTONOMOUS SYSTEMS

# The unit of the Internet: the AS




An **autonomous system** is a set of routers and networks...

managed by a **single organisation**

running under a **common routing policy** internally

has **full autonomy** over which routing protocol to run within its own network

CONCRETELY

# The AS you are inside right now

| | |
|---|---|
| AS handle | **AS9432** |
| ASN name | CANTERBURY-AS |
| organisation | University of Canterbury |
| registry | APNIC (Asia-Pacific) |
| IPv4 prefixes | 132.181.0.0/16<br>202.36.178.0/23 |

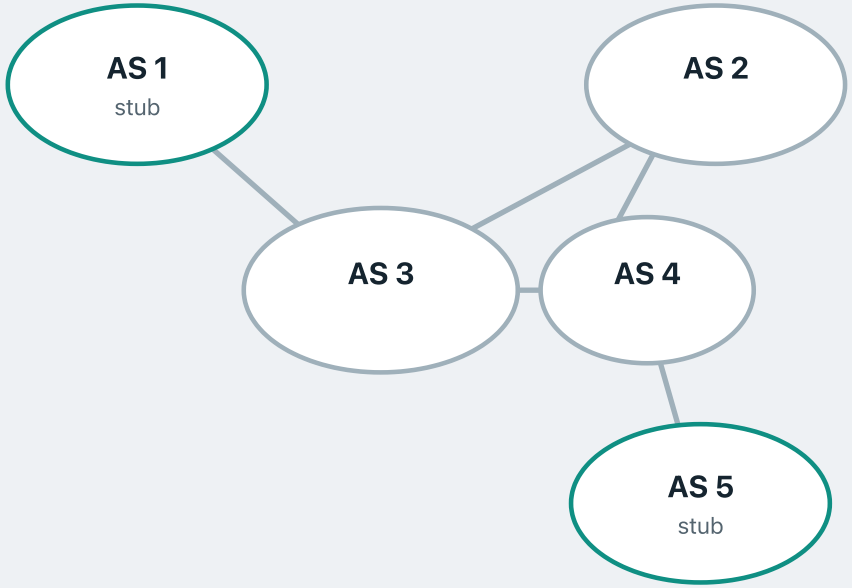
asnlookup.com, retrieved July 2026

AS9432 is the **AS number** — the handle every other AS on the Internet uses to refer to this one.

The /16 is the **subnet mask**: the first 16 bits name the network, so every 132.181.x.x address sits inside it.

TAXONOMY

# Three kinds of AS — by how traffic may flow

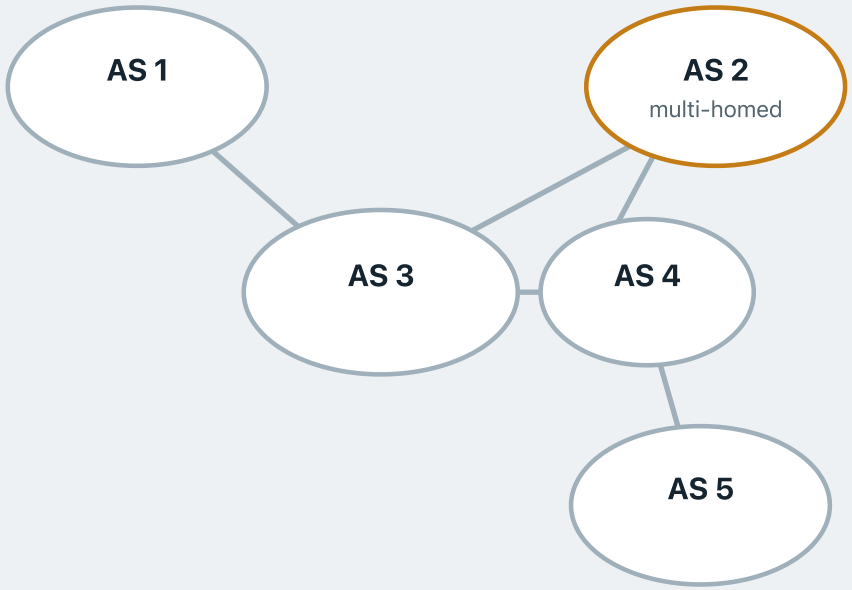



**Stub** — one connection to one **provider** (an AS that sells connectivity to other ASes).

e.g. AS 3 is the provider to AS 1, which is a stub.

TAXONOMY

# Three kinds of AS — by how traffic may flow




**Stub** — one connection to one **provider** (an AS that sells connectivity to other ASes).

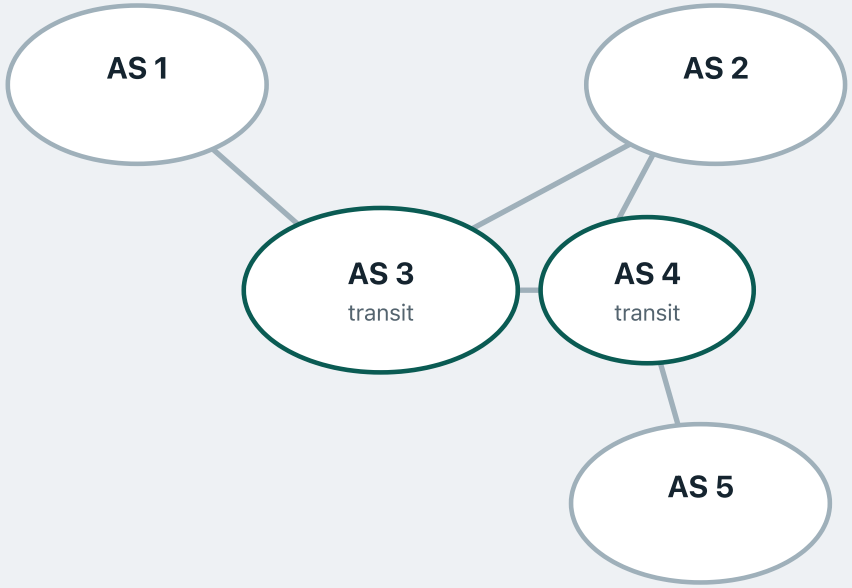
e.g. AS 3 is the provider to AS 1, which is a stub.

**Multi-homed** — several providers, but carries no through-traffic.

e.g. a datagram may not go from AS 3 through AS 2 to AS 4.

TAXONOMY

# Three kinds of AS — by how traffic may flow




**Stub** — one connection to one **provider** (an AS that sells connectivity to other ASes).

e.g. AS 3 is the provider to AS 1, which is a stub.

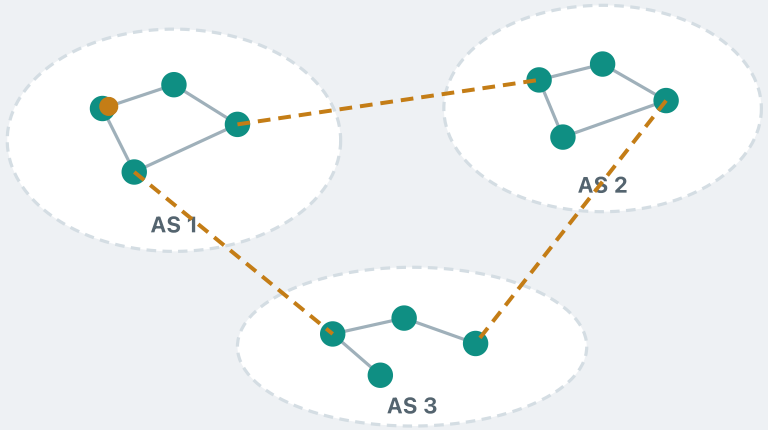
**Multi-homed** — several providers, but carries no through-traffic.

e.g. a datagram may not go from AS 3 through AS 2 to AS 4.

**Transit** — connects ASes and carries others' traffic. That service is the product. (e.g. One NZ, Verizon)

TWO ROUTING PROBLEMS

# Intra-AS routing vs Inter-AS routing




**`intra-AS routing protocol`**

Routing **within** a single AS — figure out the path a packet travels from one router to another router inside the same AS.

Each AS has **full autonomy** over which protocol to run internally.

**`inter-AS routing protocol`**

Routing **between** autonomous systems — figure out the path a packet travels from one AS to another.

REAL PROTOCOLS

# Intra-AS and Inter-AS protocols

**intra-AS routing protocols**

**RIP** (Routing Information Protocol)

Algorithm family: `distance-vector`

**OSPF** (Open Shortest Path First)

Algorithm family: **`link-state`**

**IS-IS** (Intermediate System to Intermediate System)

Algorithm family: **`link-state`**

`inter-AS routing protocols`

**BGP** (Border Gateway Protocol)

Algorithm family: `path-vector`

We will look at these protocols themselves in Module 2.

RECAP

# The routing problem — what we covered

**Routing** finds a path from every source to every destination.

It builds the **forwarding tables**; **forwarding** just reads one row and sends on.

Two families of algorithm: **link-state** and **distance-vector**.

No one controls the Internet — it is a network of **autonomous systems**.

Each AS has autonomy over its routing, so routing splits: **intra-AS** and **inter-AS**.

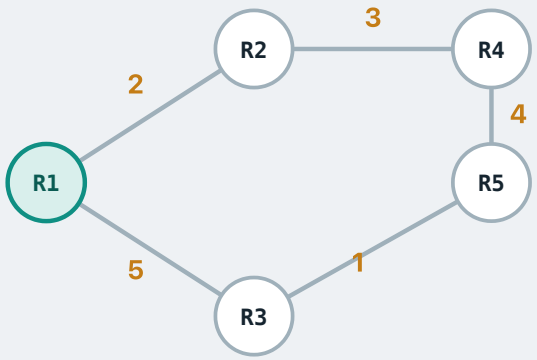
MODULE 1

# Shortest Paths & Distance-Vector Routing

1 · The Routing Problem | **2 · Shortest Paths & Distance-Vector** | 3 · Dijkstra & Link-State Routing

FOUNDATION

# Networks as graphs




Model the network as a **weighted graph**.

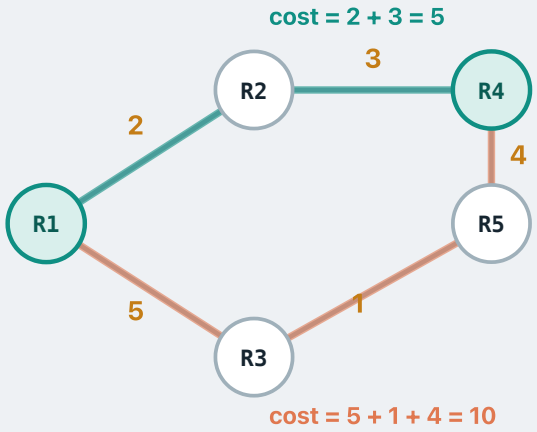
Routers are **vertices**

Links are **edges**

Link metrics are **weights / costs** — can represent inverse bandwidth, delay, or admin-assigned cost. We use these terms interchangeably: *least-cost path* = shortest path by total weight.

MOTIVATION

# Why shortest paths?




Suppose R1 wants to send a packet to R4. Two options:

**Path A:** R1 → R2 → R4 **(cost 5)**

**Path B:** R1 → R3 → R5 → R4 **(cost 10)**

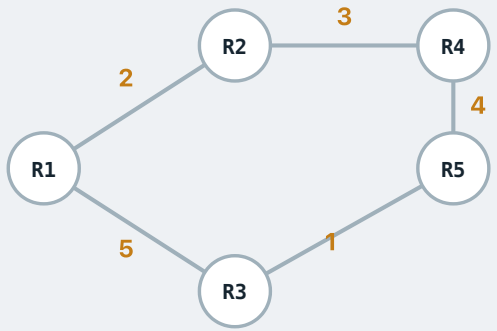
Path A wins on every count — lower delay, lower economic cost, better use of bandwidth.

COMMON ROUTING PRINCIPLE

Routing favours the **least-cost path**.

THE PROBLEM

# Computing shortest paths



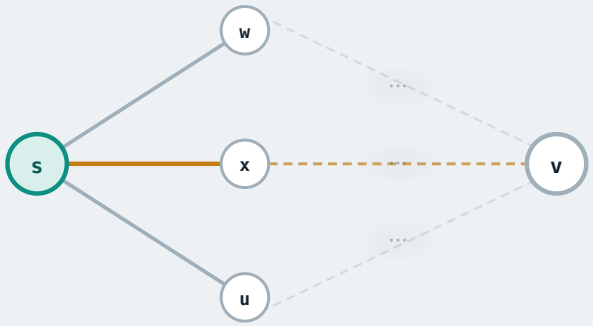

For a small network like this, we can spot the shortest paths by inspection.

But real networks have **thousands to millions** of routers — finding the shortest path for every source–destination pair is no longer obvious.

We need a systematic algorithm — a **shortest-path algorithm**.

**Relaxation** is the core building block of the common shortest-path algorithms.

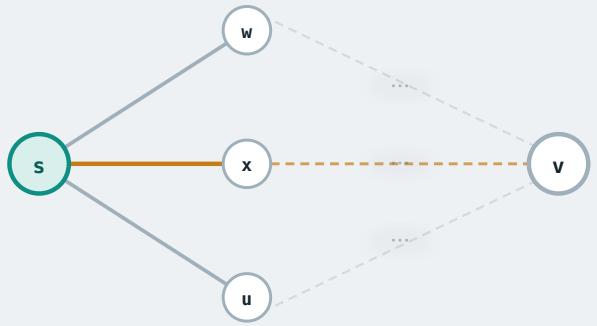
# The core operation: relaxation




**Goal:** find the shortest path from **s** to **v**.

> To reach v, s has to go through **one of its neighbours** — say **x**. What we care about is the cost of each path, so we ask: **if we pick the first hop to be x**, what would be the **least possible cost** of the path?

# The core operation: relaxation

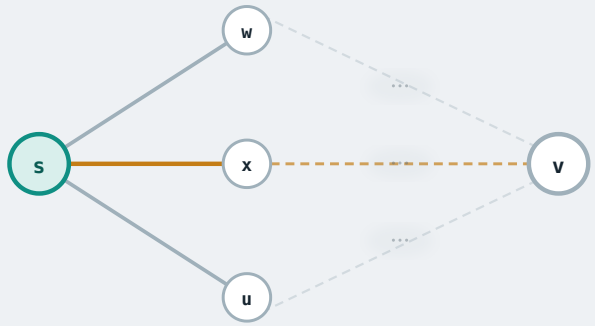



**Goal:** find the shortest path from **s** to **v**.

LEAST COST THROUGH X

The answer is quite simple: that least cost must be the **cost of the link to x**, plus the **least possible cost** from x onward to v.

# The core operation: relaxation



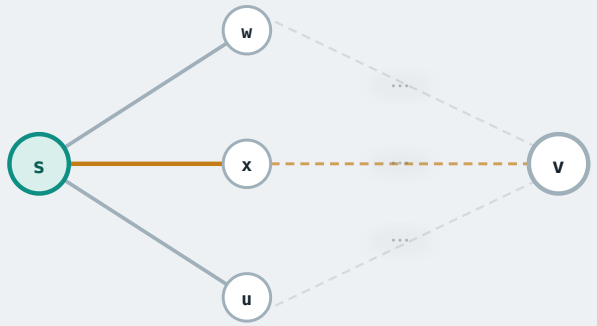

**Goal:** find the shortest path from **s** to **v**.

> From here on we write these the way the **notation** under the graph does. Three symbols, one at a time.

NOTATION

- $c(s,x)$ — cost of the link s → x
- $d_s(\cdot)$ — s's **distance vector**: one estimate per destination
- $next_s(v)$ — s's next hop towards v

# The core operation: relaxation




**Goal:** find the shortest path from **s** to **v**.

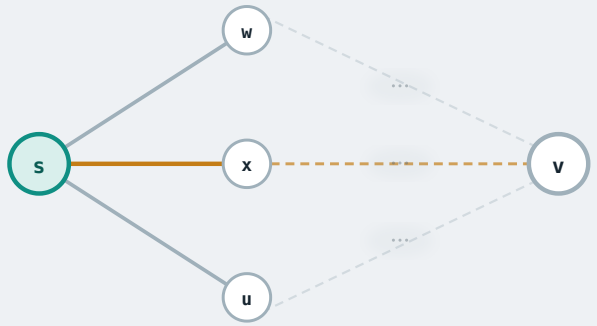
THE LINK COST

`c(s,x)` is the cost of crossing **one edge** — here, the link from s to its neighbour x.

NOTATION

- `c(s,x)` — cost of the link s → x
- $d_s(\cdot)$ — s's **distance vector**: one estimate per destination
- $next_s(v)$ — s's next hop towards v

# The core operation: relaxation




**Goal:** find the shortest path from **s** to **v**.

THE DISTANCE VECTOR, AND THE NEXT HOP

- $d_s(\cdot)$ is s's **distance vector**: a vector of **best known** distances from node s to every destination.
- e.g. $d_s(v)$ is the best known cost from s to v,
- and $d_x(v)$ is the best known cost from x to v.

$next_s(v)$ is the **next hop** that goes with $d_s(v)$: the neighbour on that best known path.

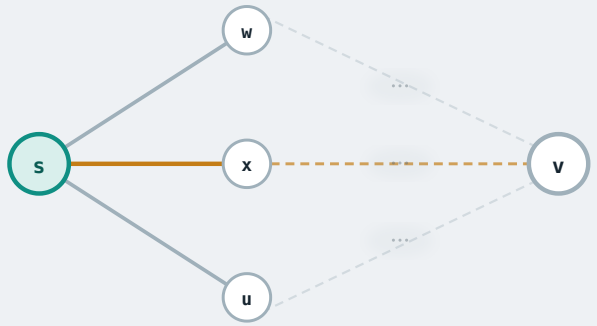
NOTATION

$c(s,x)$ — cost of the link s → x

$d_s(\cdot)$ — s's **distance vector**: one estimate per destination

$next_s(v)$ — s's next hop towards v

# The core operation: relaxation




| dest | $next_s(\cdot)$ | $d_s(\cdot)$ |
|---|---|---|
| x | x | 20 |
| v | nil | ∞ |
| … | … | … |

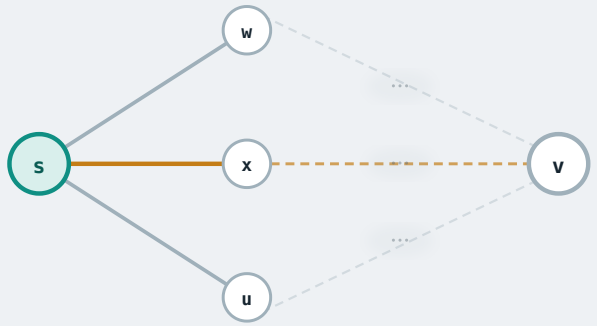
**Goal:** find the shortest path from **s** to **v**.

A CONCRETE EXAMPLE

Initially, for s — destinations x, v, and so on:

- **x**: only the direct edge is known, so take it. $d_s(x) = c(s,x) = 20$, next hop x.
- **v**: no path known yet. $d_s(v) = \infty$, next hop `nil`.

# The core operation: relaxation



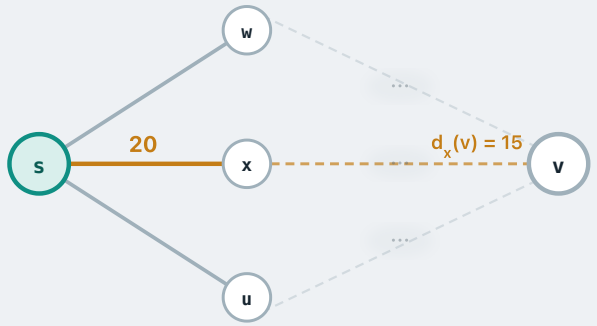

| dest | $next_s(\cdot)$ | $d_s(\cdot)$ |
|---|---|---|
| x | x | 20 |
| v | nil | ∞ |
| … | … | … |

**Goal:** find the shortest path from **s** to **v**.

LEAST COST FROM S TO V WITH FIRST HOP X

$$c(s,x) + d_x(v)$$

# The core operation: relaxation




| dest | $next_s(\cdot)$ | $d_s(\cdot)$ |
|---|---|---|
| x | x | 20 |
| v | nil | ∞ |
| … | … | … |

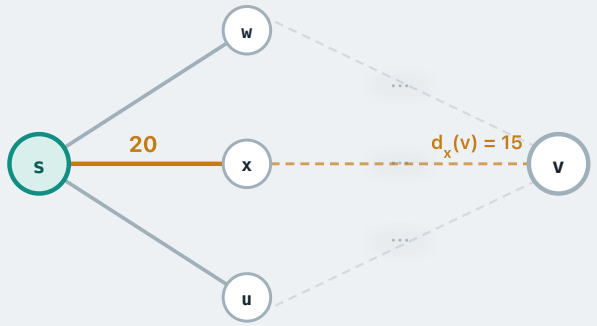
**Goal:** find the shortest path from **s** to **v**.

Ask **x** how far it is from v. It answers $d_x(v) = 15$.

So the least cost to v with **x** as the first hop is $c(s,x) + d_x(v) = 20 + 15 = \mathbf{35}$

Our best known cost to v is ∞, and going through x offers 35 — cheaper.

# The core operation: relaxation




| dest | $next_s(\cdot)$ | $d_s(\cdot)$ |
|---|---|---|
| x | x | 20 |
| v | x | 35 |
| … | … | … |

**Goal:** find the shortest path from **s** to **v**.

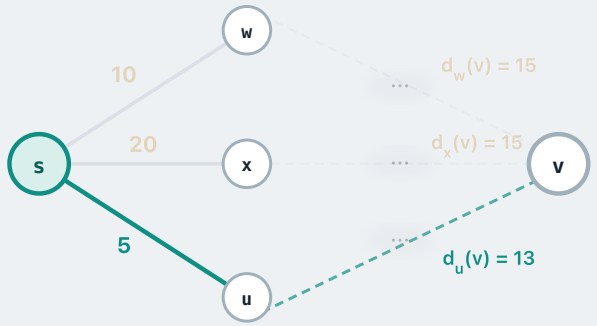
Ask **x** how far it is from v. It answers $d_x(v) = 15$.

So the least cost to v with **x** as the first hop is $c(s,x) + d_x(v) = 20 + 15 = \mathbf{35}$

Our best known cost to v is ∞, and going through x offers 35 — cheaper.

> Since it is cheaper there is no reason not to take it, so we set $next_s(v) = x$ and $d_s(v) = 35$. This is known as **relaxation**.

# The core operation: relaxation




| dest | $next_s(\cdot)$ | $d_s(\cdot)$ |
|---|---|---|
| x | x | 20 |
| v | u | 18 |
| … | … | … |

**Goal:** find the shortest path from **s** to **v**.

So far the best known cost to v is **35**, with next hop **x**.

Now on to the next neighbour, **w**.
Ask **w** how far it is from v. It answers $d_w(v) = 15$.

So the least cost to v with **w** as the first hop is $c(s,w) + d_w(v) = 10 + 15 = 25$

Our best known cost to v is 35, and going through w offers 25 — cheaper.

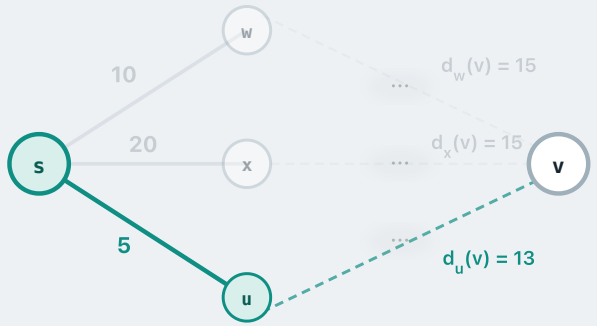
> So we take it too: $next_s(v) = w$, $d_s(v) = 25$. RELAXATION

and the other neighbour **u**: $c(s,u) + d_u(v) = 5 + 13 = 18$

18 < 25 — better still, so relax again. RELAXATION

> And update both again: $next_s(v) = u$, $d_s(v) = 18$.

# Relaxation: the formal rule




**$d_s(v)$** = best known cost s→v · **$c(s,w)$** = link cost · **$next_s(v)$** = next hop

**$d_s^*(v)$** = true optimal cost s→v

RELAXATION

Pick any neighbour **w**: if the cost to v through w beats the best known cost, take it.

```
if c(s,w) + d_w(v) < d_s(v):
  d_s(v) ← c(s,w) + d_w(v)
  next_s(v) ← w
```

OBSERVATION

- Write **$d_s^*(v)$** for the **optimal** cost from s to v — the cost of a genuinely shortest path.
- Clearly **$d_s(v) \geq d_s^*(v)$**: the estimate is always **at least** the optimal cost.
- Each relaxation can only ever **decrease** $d_s(v)$.
- So intuitively, keep relaxing for long enough and $d_s(v)$ should end up at $d_s^*(v)$.

SHORTEST-PATH ALGORITHMS

# Bellman-Ford

BELLMAN-FORD

```
Initialisation:
  d_s(s) = 0  for every node s
  d_s(u) = c(s,u)  if u is a neighbour of s, else ∞

repeat:
  for each node s:
    for each neighbour w of s:
      for each destination u:
        if c(s,w) + d_w(u) < d_s(u):  ← relax
          d_s(u) = c(s,w) + d_w(u)
          next_s(u) = w
  if no changes: stop
```

THE IDEA

Bellman–Ford uses exactly that idea, run over **every pair of nodes**: for each node s and each destination u, s keeps relaxing over its neighbours — until $d_s(u) = d_s^*(u)$ for **all** s and **all** u.

Here s and u mean *any* node and *any* destination — not the particular pair from the last slide.

WHY THIS ONE

A **distributed** variant of Bellman–Ford — which we build in a moment — is the shortest-path algorithm underneath **distance-vector routing**.

SHORTEST-PATH ALGORITHMS

# Bellman-Ford

### BELLMAN-FORD

```
Initialisation:
  d_s(s) = 0  for every node s
  d_s(u) = c(s,u)  if u is a neighbour of s, else ∞

repeat:
  for each node s:
    for each neighbour w of s:
      for each destination u:
        if c(s,w) + d_w(u) < d_s(u):  ← relax
          d_s(u) = c(s,w) + d_w(u)
          next_s(u) = w
  if no changes: stop
```

### INITIALISATION

Every node starts with what it can see for itself: cost **0** to itself, the **link cost** to each direct neighbour, and **∞** to everyone it has not heard of yet.

SHORTEST-PATH ALGORITHMS

# Bellman-Ford

### BELLMAN-FORD

```
1  Initialisation:
2    d_s(s) = 0  for every node s
3    d_s(u) = c(s,u)  if u is a neighbour of s, else ∞
4
5  repeat:
6    for each node s:
7      for each neighbour w of s:
8        for each destination u:
9          if c(s,w) + d_w(u) < d_s(u):  ← relax
10           d_s(u) = c(s,w) + d_w(u)
11           next_s(u) = w
12   if no changes: stop
```

### RELAX, AND REPEAT

Every node s performs **relaxation** for every destination u, through every neighbour w.

They keep doing this until the network **converges** — a full pass where nothing changes.

After convergence, every node s has the shortest distance $d_s(u)$ and next hop $next_s(u)$ to every other node u.

Every node, to every other node — so Bellman–Ford solves the **all-pairs** shortest-path problem, not just one source's.

# Bellman-Ford

**BELLMAN-FORD**

```
1  Initialisation:
2    d_s(s) = 0   for every node s
3    d_s(u) = c(s,u)   if u is a neighbour of s, else ∞
4
5  repeat:
6    for each node s:   ← distribute this loop
7      for each neighbour w of s:
8        for each destination u:
9          if c(s,w) + d_w(u) < d_s(u):   ← relax
10           d_s(u) = c(s,w) + d_w(u)
11           next_s(u) = w
12   if no changes: stop
```

So we now have a shortest-path algorithm. How does **distance-vector routing** use it?

Not in this shape — it rests on one **observation** about the inner loop.

**IMPORTANT OBSERVATION**

To run the inner relaxation (lines 7–11) for itself, node s only needs its **own link costs** c(s,w) and its **neighbours' distance vectors** $d_w(u)$. That is purely **local** information — s never has to learn, for example, the cost of a link **two hops away**.

# Bellman-Ford

BELLMAN-FORD

```
1  Initialisation:
2    d_s(s) = 0  for every node s
3    d_s(u) = c(s,u)  if u is a neighbour of s, else ∞
4
5  repeat:
6    for each node s:  ← distribute this loop
7      for each neighbour w of s:
8        for each destination u:
9          if c(s,w) + d_w(u) < d_s(u):  ← relax
10           d_s(u) = c(s,w) + d_w(u)
11           next_s(u) = w
12   if no changes: stop
```

**MAKING IT A DISTRIBUTED ALGORITHM**

That observation lets us break up the outer loop (line 6). Instead of **one** algorithm working through the nodes in turn, hand **every node its own copy** and let it run the inner relaxation for itself.

# Bellman-Ford

## DISTRIBUTED BELLMAN-FORD

```
1  Initialisation (at node s):
2    d_s(s) = 0
3    d_s(u) = c(s,u)  if u is a neighbour, else ∞
4
5  repeat:
6    for each neighbour w of s:
7      for each destination u:
8        if c(s,w) + d_w(u) < d_s(u):  ← relax
9          d_s(u) = c(s,w) + d_w(u)
10         next_s(u) = w
11   if any d_s(u) changed:
12     send d_s(u) to all neighbours
```

### MAKING IT A DISTRIBUTED ALGORITHM

Here is how it looks — the outer loop is gone, and the initialisation is now something **every node does for itself**.

The idea works. But it needs **one refinement**.

# Bellman-Ford

### DISTRIBUTED BELLMAN-FORD

```
1  Initialisation (at node s):
2    d_s(s) = 0
3    d_s(u) = c(s,u)  if u is a neighbour, else ∞
4
5  repeat:
6    for each neighbour w of s:
7      for each destination u:
8        if c(s,w) + d_w(u) < d_s(u):   ← relax
9          d_s(u) = c(s,w) + d_w(u)
10         next_s(u) = w
11   if any d_s(u) changed:
12     send d_s(u) to all neighbours
```

**WHAT DISTRIBUTION ADDS: LINES 11–12**

Once distributed, node s can no longer *read* its neighbours' estimates directly — there is no central shared memory holding all those values.

Therefore, whenever its own estimates change, s must **send** them to its neighbours.

This is the bare-bone algorithm of the **distance-vector protocol**.

# The distance-vector protocol

```
Initialisation (at node s):
  d_s(s) = 0
  d_s(u) = c(s,u)  if u is a neighbour, else ∞
  next_s(u) = u    if u is a neighbour, else nil
  d_w(u) = ∞  for every neighbour w, every destination u
    ← the last vector received from w, stored at s

on receiving a vector, or a link cost change:
  for each destination u:
    for each neighbour w of s:
      if c(s,w) + d_w(u) < d_s(u):  ← relax
        d_s(u) = c(s,w) + d_w(u)
        next_s(u) = w
  if any d_s(u) changed:
    send d_s(u) to all neighbours
```

For each destination u, router s keeps three things:

- its **distance vector** entry $d_s(u)$ — the current cost estimate
- the **next hop** $next_s(u)$ on that path
- each neighbour's **last advertised cost** to that destination, $d_w(u)$

# The distance-vector protocol

```
1  Initialisation (at node s):
2    d_s(s) = 0
3    d_s(u) = c(s,u)  if u is a neighbour, else ∞
4    next_s(u) = u    if u is a neighbour, else nil
5    d_w(u) = ∞  for every neighbour w, every destination u
       ← the last vector received from w, stored at s
6
7  on receiving a vector, or a link cost change:
8    for each destination u:
9      for each neighbour w of s:
10       if c(s,w) + d_w(u) < d_s(u):  ← relax
11         d_s(u) = c(s,w) + d_w(u)
12         next_s(u) = w
13   if any d_s(u) changed:
14     send d_s(u) to all neighbours
```

Recompute whenever a **vector arrives** from a neighbour, or a **local link cost changes**.

Then apply **relaxation**: for each destination, check if a neighbour offers a cheaper path.

# The distance-vector protocol

```
1  Initialisation (at node s):
2    d_s(s) = 0
3    d_s(u) = c(s,u)  if u is a neighbour, else ∞
4    next_s(u) = u    if u is a neighbour, else nil
5    d_w(u) = ∞  for every neighbour w, every destination u
       ← the last vector received from w, stored at s
6
7  on receiving a vector, or a link cost change:
8    for each destination u:
9      for each neighbour w of s:
10       if c(s,w) + d_w(u) < d_s(u):  ← relax
11         d_s(u) = c(s,w) + d_w(u)
12         next_s(u) = w
13   if any d_s(u) changed:
14     send d_s(u) to all neighbours
```

If any entry improved, send the updated vector to neighbours.

# The distance-vector protocol

```
1  Initialisation (at node s):
2    d_s(s) = 0
3    d_s(u) = c(s,u)  if u is a neighbour, else ∞
4    next_s(u) = u    if u is a neighbour, else nil
5    d_w(u) = ∞  for every neighbour w, every destination u
       ← the last vector received from w, stored at s
6
7  on receiving a vector, or a link cost change:
8    for each destination u:
9      q = next_s(u)
10     if c(s,q) + d_q(u) > d_s(u):  ← forced accept
11       d_s(u) = c(s,q) + d_q(u)
12     for each neighbour w of s:
13       if c(s,w) + d_w(u) < d_s(u):  ← relax
14         d_s(u) = c(s,w) + d_w(u)
15         next_s(u) = w
16   if any d_s(u) changed:
17     send d_s(u) to all neighbours
```

**FORCED-ACCEPT RULE**

If the update comes from your **current next-hop** for a destination, accept unconditionally — even if the cost is *worse*.

**Why?**

Your route to that destination runs **through** that neighbour. If their cost changes, yours changes with it — your old estimate is simply no longer true.

# The distance-vector protocol

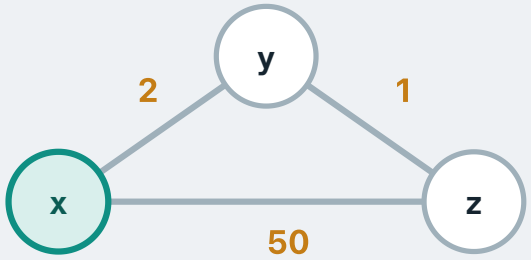
```
1  Initialisation (at node s):
2    d_s(s) = 0
3    d_s(u) = c(s,u)  if u is a neighbour, else ∞
4    next_s(u) = u    if u is a neighbour, else nil
5    d_w(u) = ∞  for every neighbour w, every destination u
       ← the last vector received from w, stored at s
6
7  on receiving a vector, or a link cost change:
8    for each destination u:
9      q = next_s(u)
10     if c(s,q) + d_q(u) > d_s(u):  ← forced accept
11       d_s(u) = c(s,q) + d_q(u)
12     for each neighbour w of s:
13       if c(s,w) + d_w(u) < d_s(u):  ← relax
14         d_s(u) = c(s,w) + d_w(u)
15         next_s(u) = w
16   if any d_s(u) changed:
17     send d_s(u) to all neighbours
```

Lines 9–15 can be simplified into the following two lines of code:

> $d_s(u) = \min_w \{ c(s,w) + d_w(u) \}$
> $next_s(u) =$ the w achieving the min

This is the form the textbook uses. We spelled it out to make the rule visible.

# Distance-vector step-by-step




**Initial:** each router knows only its own direct link costs.

**x's table**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| y | 2 | y |
| z | 50 | z |

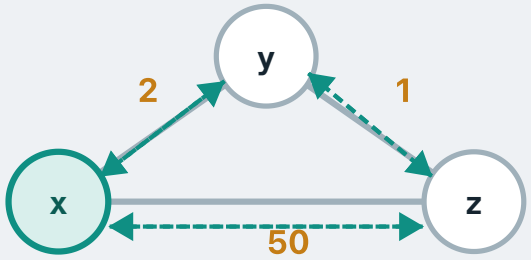
**y's table**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| x | 2 | x |
| z | 1 | z |

**z's table**

| DEST | $D_Z(\cdot)$ | $NEXT_Z(\cdot)$ |
|---|---|---|
| x | 50 | x |
| y | 1 | y |

# Distance-vector step-by-step




**Round 1:** every router sends its distance vector to each of its neighbours.

**x's table**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| y | 2 | y |
| z | 50 | z |

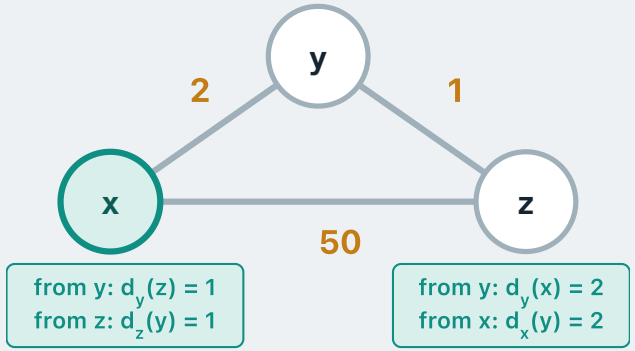
**y's table**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| x | 2 | x |
| z | 1 | z |

**z's table**

| DEST | $D_Z(\cdot)$ | $NEXT_Z(\cdot)$ |
|---|---|---|
| x | 50 | x |
| y | 1 | y |

# Distance-vector step-by-step




So each router receives **one vector per neighbour**: x gets one from y and one from z, z gets one from y and one from x, y gets one from x and one from z.

**x's table**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| y | 2 | y |
| z | 50 | z |

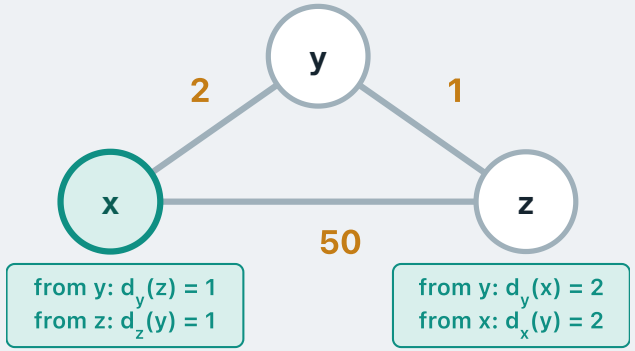
**y's table**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| x | 2 | x |
| z | 1 | z |

**z's table**

| DEST | $D_Z(\cdot)$ | $NEXT_Z(\cdot)$ |
|---|---|---|
| x | 50 | x |
| y | 1 | y |

# Distance-vector step-by-step




**X RELAXES FOR DESTINATION Z**

via y: $c(x,y) + d_y(z) = 2 + 1 = \mathbf{3}$, and $3 < 50$ — so **relax**: $d_x(z) = 3$, $next_x(z) = y$.

**x's table**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| y | 2 | y |
| z | 3 | y |

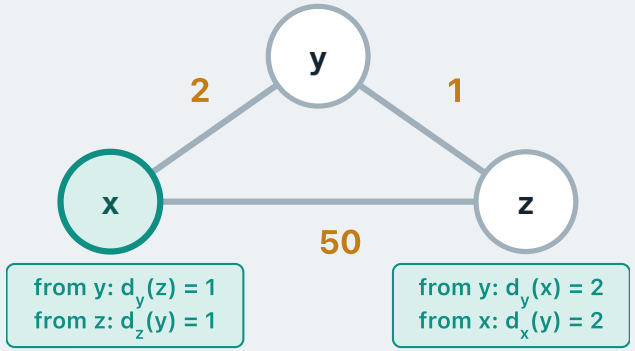
**y's table**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| x | 2 | x |
| z | 1 | z |

**z's table**

| DEST | $D_Z(\cdot)$ | $NEXT_Z(\cdot)$ |
|---|---|---|
| x | 50 | x |
| y | 1 | y |

# Distance-vector step-by-step




**X RELAXES FOR DESTINATION Y**

via z: $c(x,z) + d_z(y) = 50 + 1 = 51$, worse than the **2** x already has — **no change**.

**x's table**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| y | 2 | y |
| z | 3 | y |

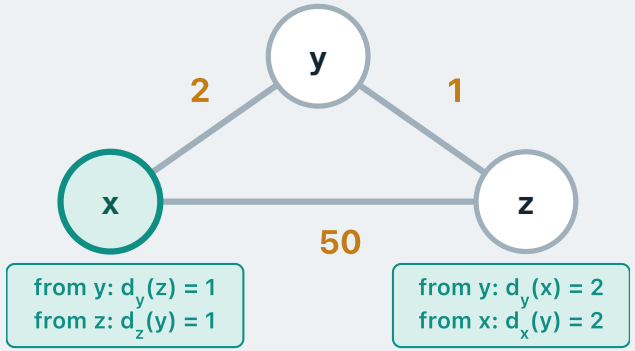
**y's table**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| x | 2 | x |
| z | 1 | z |

**z's table**

| DEST | $D_Z(\cdot)$ | $NEXT_Z(\cdot)$ |
|---|---|---|
| x | 50 | x |
| y | 1 | y |

# Distance-vector step-by-step




**Z RELAXES FOR DESTINATION X**

via y: $c(z,y) + d_y(x) = 1 + 2 = \mathbf{3}$, and $3 < 50$ — so **relax**: $d_z(x) = 3$, $next_z(x) = y$.

**x's table**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| y | 2 | y |
| z | 3 | y |

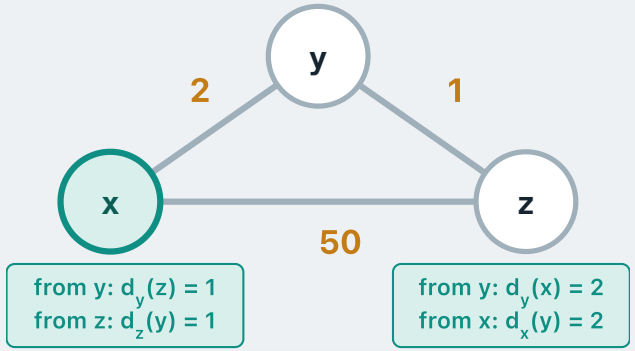
**y's table**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| x | 2 | x |
| z | 1 | z |

**z's table**

| DEST | $D_Z(\cdot)$ | $NEXT_Z(\cdot)$ |
|---|---|---|
| x | 3 | y |
| y | 1 | y |

# Distance-vector step-by-step




**Z RELAXES FOR DESTINATION Y**

via x: $c(z,x) + d_x(y) = 50 + 2 = 52$, worse than the **1** it already has — **no change**. y is already direct to both, so it changes nothing either.

**x's table**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| y | 2 | y |
| z | 3 | y |

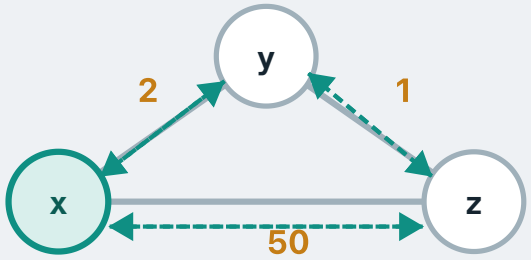
**y's table**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| x | 2 | x |
| z | 1 | z |

**z's table**

| DEST | $D_Z(\cdot)$ | $NEXT_Z(\cdot)$ |
|---|---|---|
| x | 3 | y |
| y | 1 | y |

# Distance-vector step-by-step




**Round 2:** everyone sends their updated vectors again.

**x's table**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| y | 2 | y |
| z | 3 | y |

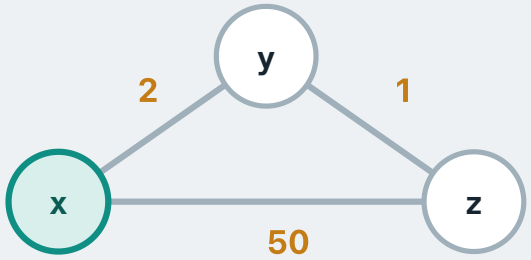
**y's table**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| x | 2 | x |
| z | 1 | z |

**z's table**

| DEST | $D_Z(\cdot)$ | $NEXT_Z(\cdot)$ |
|---|---|---|
| x | 3 | y |
| y | 1 | y |

# Distance-vector step-by-step




This time **nothing improves anywhere** — a full round with no change. The protocol has converged.

**x's table** ✓

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| y | 2 | y |
| z | 3 | y |

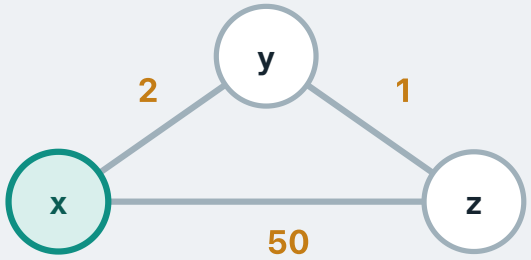
**y's table** ✓

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| x | 2 | x |
| z | 1 | z |

**z's table** ✓

| DEST | $D_Z(\cdot)$ | $NEXT_Z(\cdot)$ |
|---|---|---|
| x | 3 | y |
| y | 1 | y |

# Distance-vector step-by-step




Each router reads **$next_s(u)$** — the next-hop column — as its **forwarding table**.

**x's forwarding table**

| DEST | $NEXT_X(\cdot)$ |
|---|---|
| y | y |
| z | y |

**y's forwarding table**

| DEST | $NEXT_Y(\cdot)$ |
|---|---|
| x | x |
| z | z |

**z's forwarding table**

| DEST | $NEXT_Z(\cdot)$ |
|---|---|
| x | y |
| y | y |

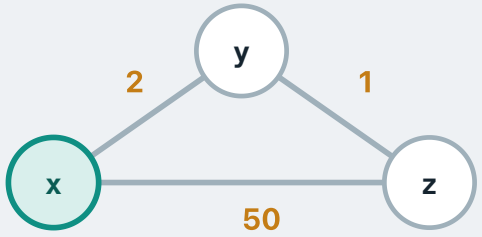
# The count-to-infinity problem

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| y | 2 | y |
| z | 3 | y |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| x | 2 | x |
| z | 1 | z |




The **converged** tables from the previous slide, after the protocol has run to completion.

DISTANCE-VECTOR UNDER FAILURE

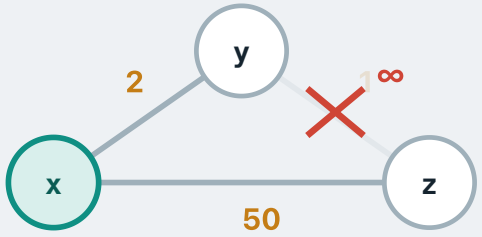
# The count-to-infinity problem

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| y | 2 | y |
| z | 3 | y |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| x | 2 | x |
| z | 1 | z |




**THE Y–Z LINK FAILS**

c(y,z) becomes ∞.

DISTANCE-VECTOR UNDER FAILURE

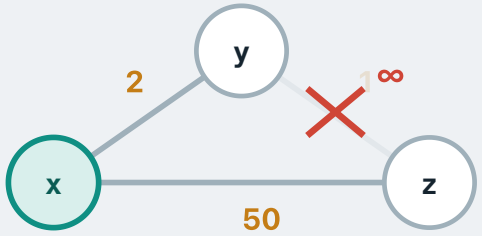
# The count-to-infinity problem

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| y | 2 | y |
| z | 3 | y |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| x | 2 | x |
| z | 1 | z |




**THIS TRIGGERS RELAXATION**

The protocol reruns whenever a vector arrives **or a link cost changes**. y's own link just changed, so y redoes its relaxation for z.

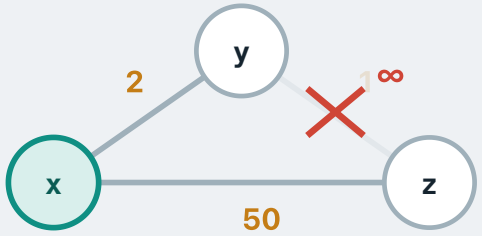
# The count-to-infinity problem

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| y | 2 | y |
| z | 3 | y |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| x | 2 | x |
| z | ∞ | z |




**Y RECOMPUTES $D_Y(Z)$**

By the protocol, y checks first whether a **forced accept** is needed.

```
 9   q = nexts(u)
10   if c(s,q) + dq(u) > ds(u):  ← forced accept
11     ds(u) = c(s,q) + dq(u)
```

$q = next_y(z) = z$, and the route through z has just become worse:

$c(y,z) + d_z(z) = \infty + 0 = \infty$, which is **>** $d_y(z) = 1$

The condition holds, so y has to accept the worse cost.

**$d_y(z) = \infty$**

DISTANCE-VECTOR UNDER FAILURE

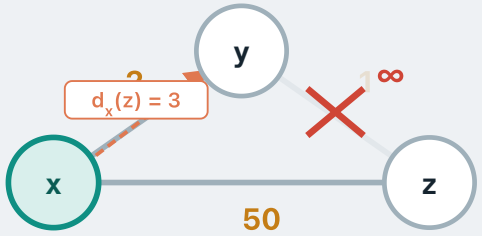
# The count-to-infinity problem

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| y | 2 | y |
| z | 3 | y |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| x | 2 | x |
| z | 5 | x |




**Y RELAXES THROUGH ITS NEIGHBOURS**

$d_y(z)$ is now ∞. y runs the relaxation loop.

```
12  for each neighbour w of s:
13    if c(s,w) + d_w(u) < d_s(u):  ← relax
14      d_s(u) = c(s,w) + d_w(u)
15      next_s(u) = w
```

through x: $c(y,x) + d_x(z) = 2 + \mathbf{3} = \mathbf{5}$

y finds $d_x(z) = 3$ from x's earlier advertisement, stored locally at y.

**$5 < \infty \rightarrow d_y(z) = 5$, $next_y(z) = x$**

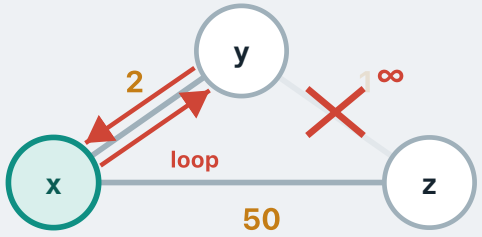
# The count-to-infinity problem

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| y | 2 | y |
| z | 3 | y |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| x | 2 | x |
| z | 5 | x |




**Y RELAXES THROUGH ITS NEIGHBOURS**

$d_y(z)$ is now ∞. y runs the relaxation loop.

```
12  for each neighbour w of s:
13    if c(s,w) + d_w(u) < d_s(u):  ← relax
14      d_s(u) = c(s,w) + d_w(u)
15      next_s(u) = w
```

through x: $c(y,x) + d_x(z) = 2 + \mathbf{3} = \mathbf{5}$

y finds $d_x(z) = 3$ from x's earlier advertisement, stored locally at y.

**$5 < \infty \rightarrow d_y(z) = 5$, $next_y(z) = x$**

> y has no idea it just created a **loop**. x's route to z runs through y, and y's route to z now runs through x, so the two of them will hand the same packet back and forth. Neither can see it: each only ever sees a cost, never the path behind it.

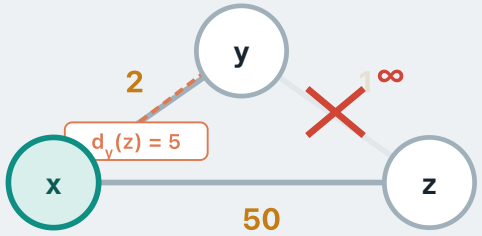
# The count-to-infinity problem

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| y | 2 | y |
| z | 3 | y |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| x | 2 | x |
| z | 5 | x |




**Y'S VECTOR CHANGED, SO Y BROADCASTS IT**

The protocol sends the vector to all neighbours whenever an entry changes. x is y's neighbour, so x receives $d_y(z) = 5$.

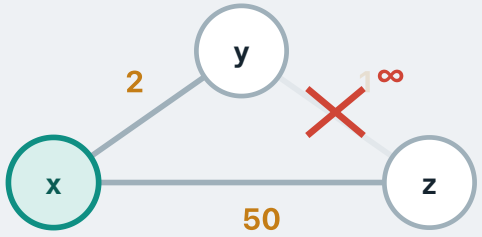
# The count-to-infinity problem

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| y | 2 | y |
| z | 3 | y |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| x | 2 | x |
| z | 5 | x |




**Y'S VECTOR CHANGED, SO Y BROADCASTS IT**

The protocol sends the vector to all neighbours whenever an entry changes. x is y's neighbour, so x receives $d_y(z) = 5$.

A vector arriving **triggers relaxation at x**. y is x's current next hop to z, so x must **forced-accept** it, worse or not:

$d_x(z) = c(x,y) + d_y(z) = 2 + 5 = \mathbf{7}$

DISTANCE-VECTOR UNDER FAILURE

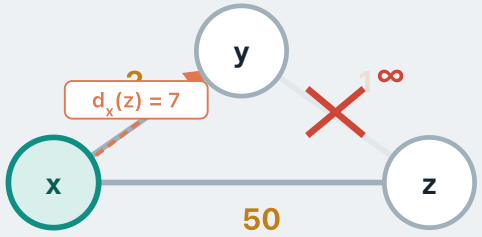
# The count-to-infinity problem

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| y | 2 | y |
| z | 7 | y |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| x | 2 | x |
| z | 5 | x |




**Y'S VECTOR CHANGED, SO Y BROADCASTS IT**

The protocol sends the vector to all neighbours whenever an entry changes. x is y's neighbour, so x receives $d_y(z) = 5$.

A vector arriving **triggers relaxation at x**. y is x's current next hop to z, so x must **forced-accept** it, worse or not:

$d_x(z) = c(x,y) + d_y(z) = 2 + 5 = \mathbf{7}$

x then runs the **relaxation loop** over its neighbours.

through z: $c(x,z) + d_z(z) = 50 + 0 = \mathbf{50}$

50 is not better than 7, so the direct link does not win.

**$d_x(z) = 7$, $next_x(z) = y$**

x's own distance vector has changed, so it must advertise that change to its neighbours — in particular, it sends **y** this update.

DISTANCE-VECTOR UNDER FAILURE

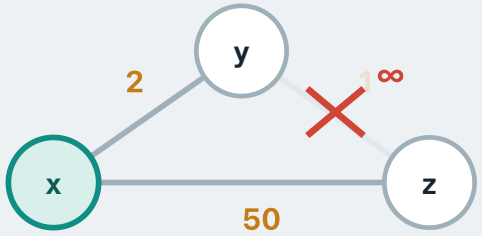
# The count-to-infinity problem

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| y | 2 | y |
| z | 7 | y |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| x | 2 | x |
| z | 5 | x |




X'S VECTOR CHANGED, SO X BROADCASTS IT

y receives $d_x(z) = 7$, which **triggers relaxation at y**.

DISTANCE-VECTOR UNDER FAILURE

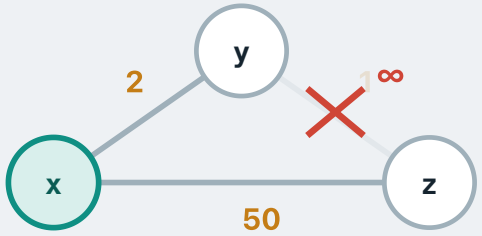
# The count-to-infinity problem

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| y | 2 | y |
| z | 7 | y |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| x | 2 | x |
| z | 9 | x |




X'S VECTOR CHANGED, SO X BROADCASTS IT

y receives $d_x(z) = 7$, which **triggers relaxation at y**.

x is now y's current next hop to z, so y must **forced-accept** in turn:

$d_y(z) = c(y,x) + d_x(z) = 2 + 7 = \mathbf{9}$

y then runs the **relaxation loop** over its neighbours.

through z: $c(y,z) + d_z(z) = \infty + 0 = \infty$

Still nothing better.

**$d_y(z) = 9$, $next_y(z) = x$**

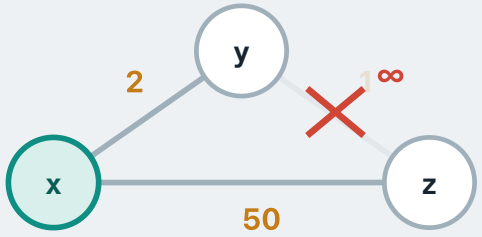
# The count-to-infinity problem

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| y | 2 | y |
| z | 50 | z |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| x | 2 | x |
| z | 52 | x |




**THE LOOP KEEPS GOING**

y sends 9 to x, x computes 2+9 = **11**, y: 2+11 = **13**, x: 2+13 = **15** …

The climb only stops when going via the other router costs more than the **direct x–z link of 50**.

> x switches to the direct link at $\mathbf{d_x(z) = 50}$, and y follows at $\mathbf{d_y(z) = 52}$. The network is stable again.

DISTANCE-VECTOR UNDER FAILURE

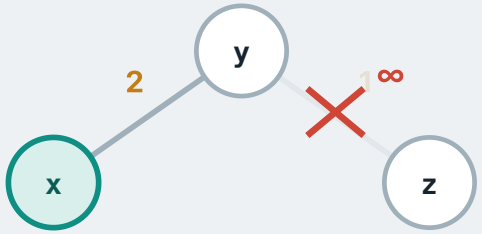
# The count-to-infinity problem

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| y | 2 | y |
| z | 50 | z |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| x | 2 | x |
| z | 52 | x |




### THE LOOP KEEPS GOING

y sends 9 to x, x computes 2+9 = **11**, y: 2+11 = **13**, x: 2+13 = **15** …

The climb only stops when going via the other router costs more than the **direct x–z link of 50**.

x switches to the direct link at $\mathbf{d_x(z) = 50}$, and y follows at $\mathbf{d_y(z) = 52}$. The network is stable again.

And it is **slow**. The cost only climbs two at a time, so it takes roughly **22 rounds** of exchanges before the direct link finally wins. For all of that time x and y are still handing the same packets back and forth.

DISTANCE-VECTOR UNDER FAILURE

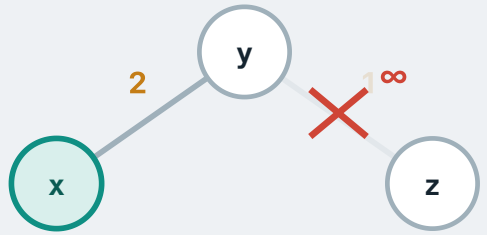
# The count-to-infinity problem

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| y | 2 | y |
| z | … | y |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| x | 2 | x |
| z | … | x |




But what if there were **no direct link** between x and z?

The costs would never stop growing — they would count all the way to ∞.

> This is the **count-to-infinity problem**: in **distance-vector routing**, a router cannot tell that a neighbour's route runs back through itself, so the loop forms and the cost climbs one step at a time.

How do we address this?

FIXING DV

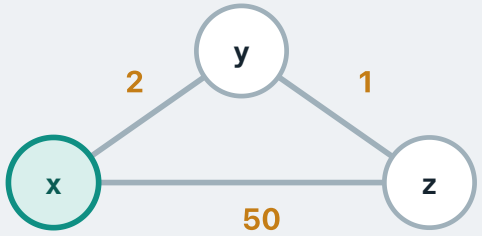
# Poison reverse

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| z | 50 | z |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| z | 1 | z |




**Rule:** when node s hears an advertisement from neighbour w, adopts the route, and sets w as its next hop — s **lies** to w by advertising ∞ for that destination, while reporting the true cost to all other neighbours.

Lying = setting the distance vector to ∞. This actively warns w: "don't use me to reach this destination."

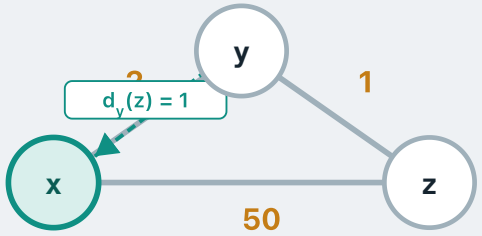
# Poison reverse

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| z | 50 | z |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| z | 1 | z |




y sends its distance vector to x, advertising $d_y(z) = 1$.

x receives this and can now consider a route to z via y.

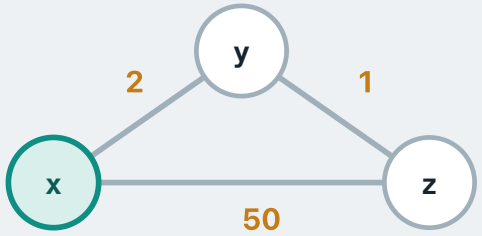
# Poison reverse

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| z | 3 | y |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| z | 1 | z |




x performs relaxation:

via y: $c(x,y) + d_y(z) = 2 + 1 = \mathbf{3}$
direct to z: cost = **50**

3 < 50, so x adopts the route via y:

**$d_x(z) = 3$, $next_x(z) = y$**

FIXING DV

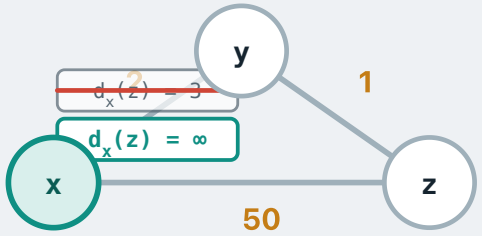
# Poison reverse

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| z | 3 | y |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| z | 1 | z |




x adopted the route to z **from y** — y is the next hop.

Poison reverse kicks in: x **lies to y**, advertising $d_x(z) = \infty$.

Without poison reverse, x would tell y its true cost $d_x(z) = 3$ — and that is what causes the count-to-infinity situation.

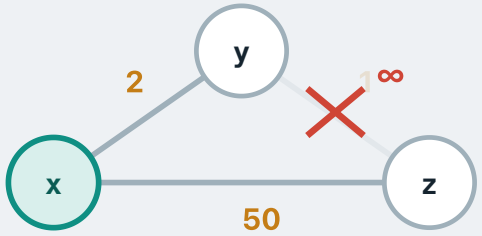
# Poison reverse

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| z | 3 | y |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| z | ∞ | nil |




The y–z link fails. y looks for an alternative route to z:

via x: $c(y,x) + d_x(z) = 2 + \infty = \infty$
(x has been poisoning y)
direct to z: link down → ∞

y has no route to z: **$d_y(z) = \infty$**

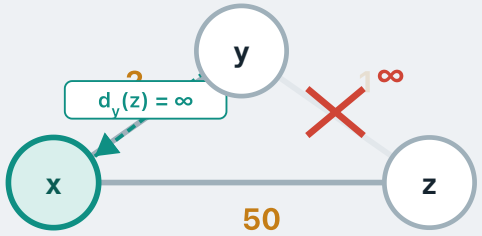
# Poison reverse

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| z | ∞ | y |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| z | ∞ | nil |




y sends $d_y(z) = \infty$ to x.

Since y is x's next-hop, x must accept (**forced-accept rule**):
$d_x(z) = \infty$.

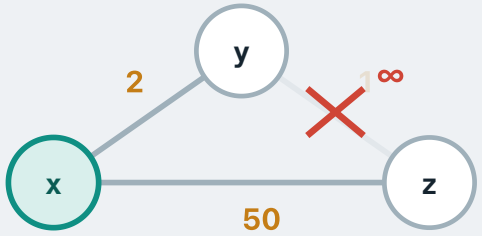
# Poison reverse

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| z | 50 | z |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| z | ∞ | nil |




y sends $d_y(z) = \infty$ to x.

Since y is x's next-hop, x must accept (**forced-accept rule**):
$d_x(z) = \infty$.

x then performs relaxation: direct to z costs **50**, cheaper than ∞.

Update: $d_x(z) = 50$, $next_x(z) = z$

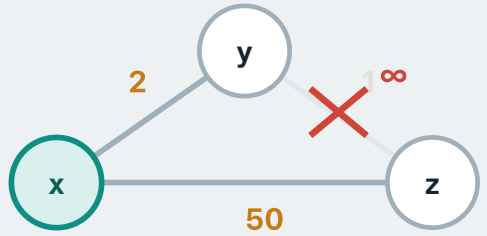
# Poison reverse

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| z | 50 | z |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| z | ∞ | nil |




**What should x advertise to y now — the true $d_x(z)$ = 50, or ∞?**

Think about whether poison reverse applies here.

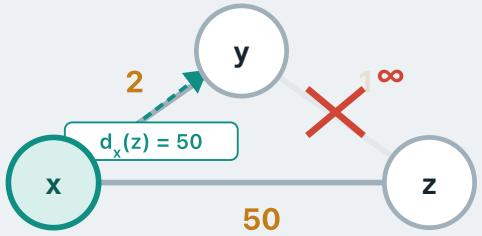
# Poison reverse

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| z | 50 | z |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| z | 52 | x |




**The true value, 50** — x's next-hop to z is now **z**, not y, so poison reverse does not apply:

$d_x(z) = 50$

y performs relaxation:

via x: $c(y,x) + d_x(z) = 2 + 50 = \mathbf{52}$
direct to z: ∞

$52 < \infty$ → update: $d_y(z) = 52$, $next_y(z) = x$

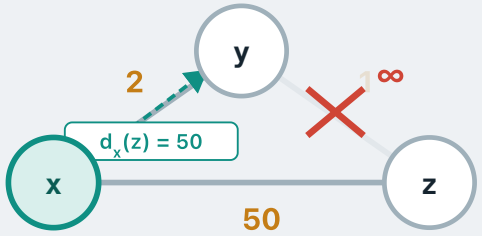
# Poison reverse

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| z | 50 | z |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| z | 52 | x |




**The network converges correctly.**

Poison reverse prevents 2-node loops.

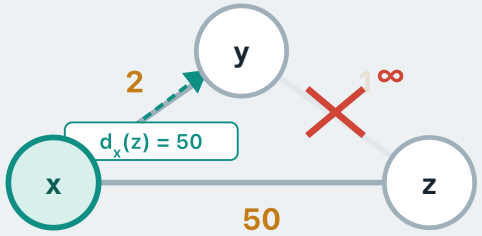
# Poison reverse

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| z | 50 | z |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| z | 52 | x |




**But does poison reverse completely solve the count-to-infinity problem?**

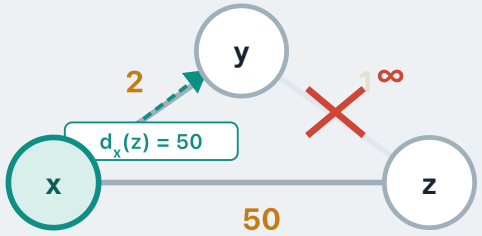
# Poison reverse

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| z | 50 | z |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| z | 52 | x |




**No.**

Can you think of a case where it fails?

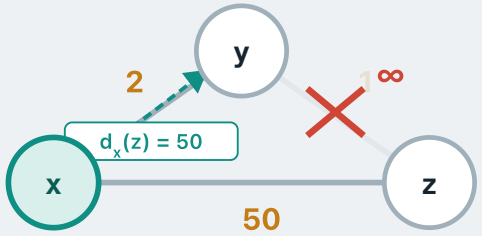
# Poison reverse

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| z | 50 | z |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| z | 52 | x |




Loops involving **three or more nodes** will not be detected by poison reverse.

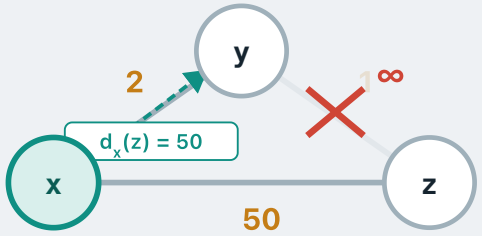
# Poison reverse

**x**

| DEST | $D_X(\cdot)$ | $NEXT_X(\cdot)$ |
|---|---|---|
| z | 50 | z |

**y**

| DEST | $D_Y(\cdot)$ | $NEXT_Y(\cdot)$ |
|---|---|---|
| z | 52 | x |




Despite this limitation, poison reverse is still used as a heuristic in distance-vector routing protocols such as **RIP**.

DISTANCE–VECTOR RECAP

# What we covered

**The goal**: routing favours the **least-cost path** — which is the shortest-path problem.

**Relaxation**: the core mechanism in shortest-path algorithms — if a neighbour offers a cheaper cost, take it, and make that neighbour your next hop.

**The distance-vector protocol**: a **distributed** Bellman–Ford. Every node performs relaxation independently and repeatedly, using **only what its direct neighbours tell it** — and passes its own result back to those same neighbours.

So DV is a purely **distributed, local** algorithm: **no router ever needs the full topology** of the network.

**Its limitation**: **count-to-infinity**. Poison reverse is a heuristic that mitigates it, but does not completely solve it.

MODULE 1

# Dijkstra's Algorithm & Link-State Routing

A faster shortest-path algorithm — and the flooding mechanism that makes it possible.

1 · The Routing Problem | 2 · Shortest Paths & Distance-Vector | **3 · Dijkstra & Link-State Routing**

RECALL

# The distance-vector approach

We have seen the **distance-vector protocol** — Bellman-Ford, distributed.

Key property: each router only talks to its **direct neighbours**. No router needs the global topology — each one performs relaxation using its own link costs and the distance vectors it receives.

But it has one limitation: **count-to-infinity**. A distance vector reports **distances, not paths**, so a router cannot see that the route it is offered runs back through itself.

Poison reverse helps, but not for loops of three or more nodes.

> **Can a different routing approach avoid this problem?**

A DIFFERENT APPROACH

# What if every router had the full map?

A simple idea: what if every router knew the **complete network topology**? Then each router could compute its shortest paths to all destinations **directly**.

This avoids DV's limitation: **no count-to-infinity** — routers share the **map**.

Each router then works out the **shortest path** over that whole topology itself — and a shortest path can never contain a loop, since going round in a circle only adds cost.

This is the idea behind **link-state routing**.

THE LINK-STATE IDEA

# How link-state routing works

**Step 1 · discover**

Each router discovers its **neighbours** and knows the **cost of each of its own links**.

**Step 2 · flood**

Each router broadcasts its local link information — a **link-state advertisement** (LSA) — to **every other router**. This process is called **flooding**.

**Step 3 · compute**

Every router assembles all LSAs into the **same complete map** and runs a **shortest-path algorithm** to build its forwarding table.

**Step 4 · re-flood**

When a link cost changes or a link fails, the affected router floods a **new LSA**. Every router updates its map and reruns the algorithm.

**Next:** Step 3 — the shortest-path algorithm. **Later:** Steps 1, 2 & 4.

THE ALGORITHM QUESTION

# What algorithm do we need?

**Goal:** given the full topology, every source node **s** needs the shortest-path cost to every other node **v**, and the **next hop** to reach it.

Bellman–Ford would do the job perfectly well — we have already seen it work, and it does not even need the full topology: the routers only ever talk to their neighbours.

But now every router *does* know the **full topology** — and in that setting there is a better-suited choice: **Dijkstra's algorithm**.

DIJKSTRA'S ALGORITHM

# How it works

```
initialise d_s(s) = 0
   d_s(v) = c(s,v) if v is a neighbour of s, else ∞
   pred_s(v) = s if v is a neighbour of s, else nil
S = {s} (settled set)

repeat:
   u = unsettled node with smallest d_s(u)
   add u to S
   for each neighbour w of u:
      if d_s(u) + c(u,w) < d_s(w): ← ???
         d_s(w) = d_s(u) + c(u,w)
         pred_s(w) = u
until all nodes settled
```

NOTATION

**$d_s(v)$** — the best cost from s to v known *so far*

**$d_s*(v)$** — the **optimal** cost from s to v (the * means optimal, as before)

**$c(u,w)$** — the cost of the link u–w

**$pred_s(v)$** — v's **predecessor**: the node we reached v from on the **current best known path** to v

e.g. if the best known path to v is `s → a → b → v`
$pred_s(v) = b$

DIJKSTRA'S ALGORITHM

# How it works

**initialise $d_s(s) = 0$**

  **$d_s(v) = c(s,v)$ if v is a neighbour of s, else $\infty$**

  **$pred_s(v) = s$ if v is a neighbour of s, else nil**

S = {s} (settled set)

**repeat:**

  u = unsettled node with smallest $d_s(u)$

  add u to S

  for each neighbour w of u:

    if $d_s(u) + c(u,w) < d_s(w)$: ← ???

      $d_s(w) = d_s(u) + c(u,w)$

      $pred_s(w) = u$

**until** all nodes settled

**START**

- **$d_s(s) = 0$** — s is zero away from itself.
- **$d_s(v) = c(s,v)$** for each neighbour v — the best cost we know so far.
- **$d_s(v) = \infty$** for everyone else — we know nothing about them yet.
- **$pred_s(v) = s$** for each neighbour v — we reach it straight from s; **nil** for everyone else, since we have no path to them yet.

DIJKSTRA'S ALGORITHM

# How it works

```
initialise d_s(s) = 0
   d_s(v) = c(s,v) if v is a neighbour of s, else ∞
   pred_s(v) = s if v is a neighbour of s, else nil
S = {s} (settled set)

repeat:
   u = unsettled node with smallest d_s(u)
   add u to S
   for each neighbour w of u:
      if d_s(u) + c(u,w) < d_s(w): ← ???
         d_s(w) = d_s(u) + c(u,w)
         pred_s(w) = u
until all nodes settled
```

**THE SETTLED SET S**

S holds the nodes whose distance is **final**: once u is in S we have $d_s(u) = d_s^*(u)$, the optimal cost.

It starts holding just **s**, whose distance is already final.

DIJKSTRA'S ALGORITHM

# How it works

```
initialise d_s(s) = 0
   d_s(v) = c(s,v) if v is a neighbour of s, else ∞
   pred_s(v) = s if v is a neighbour of s, else nil
S = {s} (settled set)

repeat:
   u = unsettled node with smallest d_s(u)
   add u to S
   for each neighbour w of u:
      if d_s(u) + c(u,w) < d_s(w): ← ???
         d_s(w) = d_s(u) + c(u,w)
         pred_s(w) = u
until all nodes settled
```

**PICK AND SETTLE**

Take the unsettled node with the **smallest current distance**.

Move it into S — its distance must **already be optimal**: **$d_s(u) = d_s*(u)$**.

**EXERCISE**

**Why is this the case?** Have a think about it offline.

DIJKSTRA'S ALGORITHM

# How it works

```
initialise d_s(s) = 0
   d_s(v) = c(s,v) if v is a neighbour of s, else ∞
   pred_s(v) = s if v is a neighbour of s, else nil
S = {s} (settled set)

repeat:
   u = unsettled node with smallest d_s(u)
   add u to S
   for each neighbour w of u:
      if d_s(u) + c(u,w) < d_s(w): ← ???
         d_s(w) = d_s(u) + c(u,w)
         pred_s(w) = u
until all nodes settled
```

**WHAT DO WE DO WITH U?**

We have just taken the unsettled node **u** with the smallest distance and moved it into S.

Now we go through **each neighbour w of u** and run the **boxed** lines on it.

**Do you recognise this operation?**

DIJKSTRA'S ALGORITHM

# How it works

```
initialise d_s(s) = 0
   d_s(v) = c(s,v) if v is a neighbour of s, else ∞
   pred_s(v) = s if v is a neighbour of s, else nil
S = {s} (settled set)

repeat:
   u = unsettled node with smallest d_s(u)
   add u to S
   for each neighbour w of u:
      if d_s(u) + c(u,w) < d_s(w): ← relaxation!
         d_s(w) = d_s(u) + c(u,w)
         pred_s(w) = u
until all nodes settled
```

**RELAXATION**

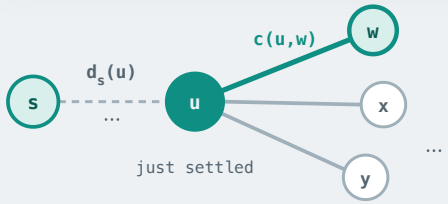
- The same core operation as in **distance vector** — but with the roles **swapped**.
- There, a node asked: *can **I** get a shorter path through one of my neighbours?*
- Here we ask: *can one of my **neighbours** get a shorter path through **me**?*

# How it works

```
initialise d_s(s) = 0
   d_s(v) = c(s,v) if v is a neighbour of s, else ∞
   pred_s(v) = s if v is a neighbour of s, else nil
S = {s} (settled set)

repeat:
   u = unsettled node with smallest d_s(u)
   add u to S
   for each neighbour w of u:
      if d_s(u) + c(u,w) < d_s(w): ← relaxation!
         d_s(w) = d_s(u) + c(u,w)
         pred_s(w) = u
until all nodes settled
```

**RELAXATION**

Hold **u** fixed and go through its neighbours in turn — say **w**. Ask: is the path to w **via u** shorter than w's current best, $\mathbf{d_s(w)}$?



DIJKSTRA'S ALGORITHM

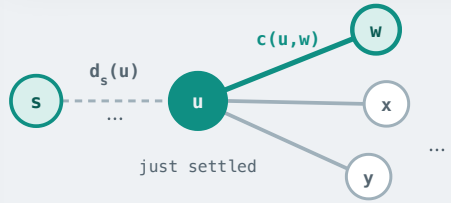
# How it works

```
initialise d_s(s) = 0
   d_s(v) = c(s,v) if v is a neighbour of s, else ∞
   pred_s(v) = s if v is a neighbour of s, else nil
S = {s} (settled set)

repeat:
   u = unsettled node with smallest d_s(u)
   add u to S
   for each neighbour w of u:
      if d_s(u) + c(u,w) < d_s(w): ← relaxation!
         d_s(w) = d_s(u) + c(u,w)
         pred_s(w) = u
until all nodes settled
```

**RELAXATION**

If it is, take it: $\mathbf{d_s(w)}$ becomes $\mathbf{d_s(u) + c(u,w)}$, and since we reached w from u, $\mathbf{pred_s(w) = u}$.

That is exactly **relaxation**.



DIJKSTRA'S ALGORITHM

# How it works

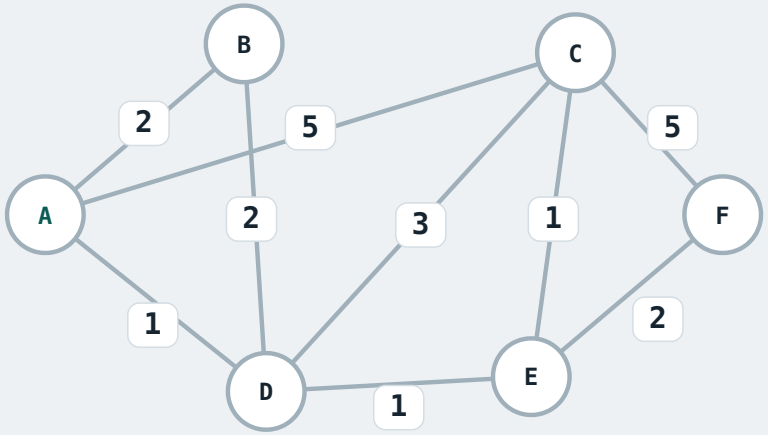
```
initialise d_s(s) = 0
   d_s(v) = c(s,v) if v is a neighbour of s, else ∞
   pred_s(v) = s if v is a neighbour of s, else nil
S = {s} (settled set)

repeat:
   u = unsettled node with smallest d_s(u)
   add u to S
   for each neighbour w of u:
      if d_s(u) + c(u,w) < d_s(w): ← relaxation!
         d_s(w) = d_s(u) + c(u,w)
         pred_s(w) = u
until all nodes settled
```

**REPEAT**

Pick the next minimum, settle it, relax outward — until every node is settled.

DIJKSTRA'S ALGORITHM

# Step-by-step walkthrough

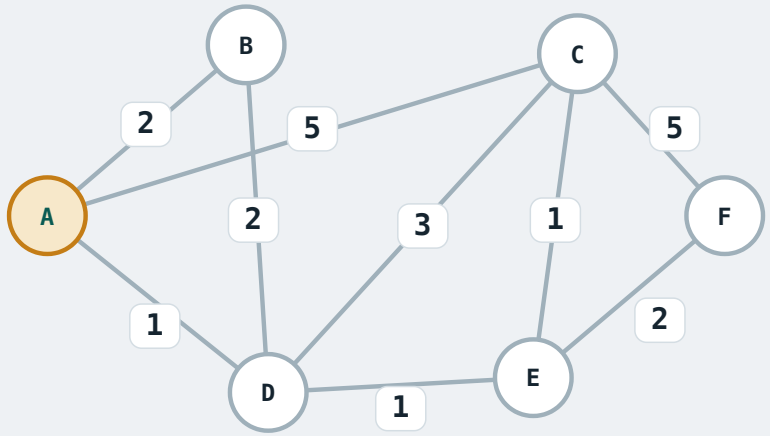



Click → to step through

| Step | S | $d_S(A)$ $pred_S(A)$ | $d_S(B)$ $pred_S(B)$ | $d_S(C)$ $pred_S(C)$ | $d_S(D)$ $pred_S(D)$ | $d_S(E)$ $pred_S(E)$ | $d_S(F)$ $pred_S(F)$ |
|---|---|---|---|---|---|---|---|

DIJKSTRA'S ALGORITHM

# Step-by-step walkthrough

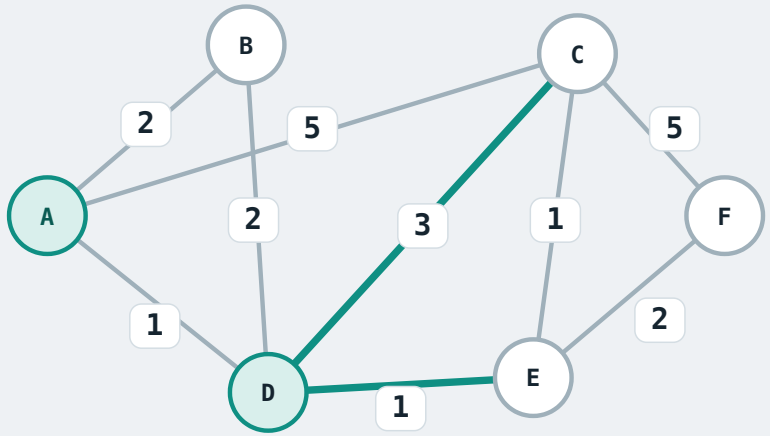



Initialisation

| Step | S | $d_S(A)$ $pred_S(A)$ | $d_S(B)$ $pred_S(B)$ | $d_S(C)$ $pred_S(C)$ | $d_S(D)$ $pred_S(D)$ | $d_S(E)$ $pred_S(E)$ | $d_S(F)$ $pred_S(F)$ |
|---|---|---|---|---|---|---|---|
| init | {A} | 0,– | 2,A | 5,A | 1,A | ∞ | ∞ |

DIJKSTRA'S ALGORITHM

# Step-by-step walkthrough




Step 1 — settle and relax

**Settle:** D has the smallest tentative distance: $d_S(D) = 1$. Add D to S → $d_S^*(D) = 1$.

**Relax D's neighbours:**

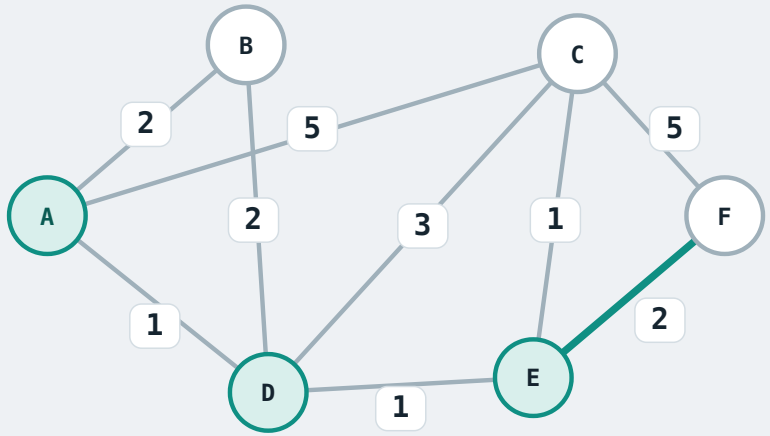
$d_S(D) + c(D,C) = 1 + 3 = 4 < 5 \rightarrow d_S(C) = 4$, $pred_S(C) = D$

$d_S(D) + c(D,E) = 1 + 1 = 2 < \infty \rightarrow d_S(E) = 2$, $pred_S(E) = D$

| Step | S | $d_S(A)$ $pred_S(A)$ | $d_S(B)$ $pred_S(B)$ | $d_S(C)$ $pred_S(C)$ | $d_S(D)$ $pred_S(D)$ | $d_S(E)$ $pred_S(E)$ | $d_S(F)$ $pred_S(F)$ |
|---|---|---|---|---|---|---|---|
| init | {A} | 0,– | 2,A | 5,A | 1,A | ∞ | ∞ |
| 1 | {A,D} | – | 2,A | **4,D** | – | **2,D** | ∞ |

DIJKSTRA'S ALGORITHM

# Step-by-step walkthrough




Step 2 — settle and relax

**Settle:** E has the smallest tentative distance: $d_s(E) = 2$. Add E to S → $d_s*(E) = 2$.

**Relax E's neighbours:**

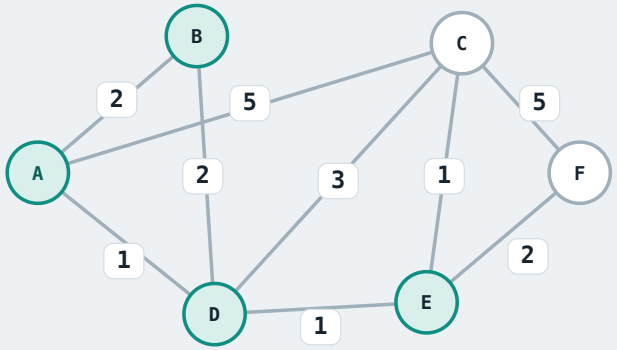
$d_s(E) + c(E,C) = 2 + 1 = 3 < 4 \rightarrow d_s(C) = 3$, $pred_s(C) = E$

$d_s(E) + c(E,F) = 2 + 2 = 4 < \infty \rightarrow d_s(F) = 4$, $pred_s(F) = E$

| Step | S | $d_s(A)$ $pred_s(A)$ | $d_s(B)$ $pred_s(B)$ | $d_s(C)$ $pred_s(C)$ | $d_s(D)$ $pred_s(D)$ | $d_s(E)$ $pred_s(E)$ | $d_s(F)$ $pred_s(F)$ |
|---|---|---|---|---|---|---|---|
| init | {A} | 0,— | 2,A | 5,A | 1,A | ∞ | ∞ |
| 1 | {A,D} | — | 2,A | **4,D** | — | **2,D** | ∞ |

DIJKSTRA'S ALGORITHM

# Step-by-step walkthrough

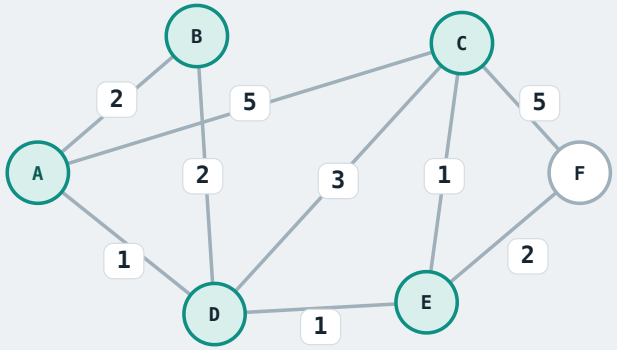



Step 3 of 5

| Step | S | $d_S(A)$ $pred_S(A)$ | $d_S(B)$ $pred_S(B)$ | $d_S(C)$ $pred_S(C)$ | $d_S(D)$ $pred_S(D)$ | $d_S(E)$ $pred_S(E)$ | $d_S(F)$ $pred_S(F)$ |
|---|---|---|---|---|---|---|---|
| init | {A} | 0,– | 2,A | 5,A | 1,A | ∞ | ∞ |
| 1 | {A,D} | – | 2,A | **4,D** | – | **2,D** | ∞ |
| 2 | {A,D,E} | – | 2,A | **3,E** | – | – | **4,E** |
| 3 | {A,D,E,B} | – | – | 3,E | – | – | 4,E |

DIJKSTRA'S ALGORITHM

# Step-by-step walkthrough

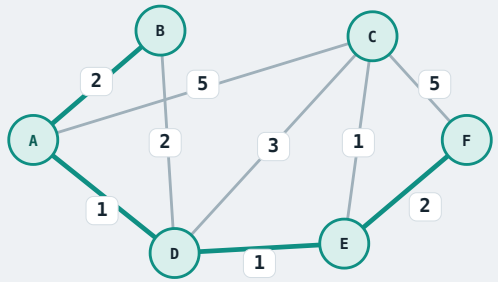



Step 4 of 5

| Step | S | $d_S(A)$ $pred_S(A)$ | $d_S(B)$ $pred_S(B)$ | $d_S(C)$ $pred_S(C)$ | $d_S(D)$ $pred_S(D)$ | $d_S(E)$ $pred_S(E)$ | $d_S(F)$ $pred_S(F)$ |
|---|---|---|---|---|---|---|---|
| init | {A} | 0,– | 2,A | 5,A | 1,A | ∞ | ∞ |
| 1 | {A,D} | – | 2,A | **4,D** | – | **2,D** | ∞ |
| 2 | {A,D,E} | – | 2,A | **3,E** | – | – | **4,E** |
| 3 | {A,D,E,B} | – | – | 3,E | – | – | 4,E |
| 4 | {A,D,E,B,C} | – | – | – | – | – | 4,E |

DIJKSTRA'S ALGORITHM

# Step-by-step walkthrough




Step 5 of 5

| Step | S | $d_S(A)$ $pred_S(A)$ | $d_S(B)$ $pred_S(B)$ | $d_S(C)$ $pred_S(C)$ | $d_S(D)$ $pred_S(D)$ | $d_S(E)$ $pred_S(E)$ | $d_S(F)$ $pred_S(F)$ |
|---|---|---|---|---|---|---|---|
| init | {A} | 0,– | 2,A | 5,A | 1,A | ∞ | ∞ |
| 1 | {A,D} | – | 2,A | **4,D** | – | **2,D** | ∞ |
| 2 | {A,D,E} | – | 2,A | **3,E** | – | – | **4,E** |
| 3 | {A,D,E,B} | – | – | 3,E | – | – | 4,E |
| 4 | {A,D,E,B,C} | – | – | – | – | – | 4,E |
| 5 | {A,D,E,B,C,F} | – | – | – | – | – | – |

DIJKSTRA'S ALGORITHM

# How do we build the forwarding table from this?

| Step | S | $d_s(A)$ $pred_s(A)$ | $d_s(B)$ $pred_s(B)$ | $d_s(C)$ $pred_s(C)$ | $d_s(D)$ $pred_s(D)$ | $d_s(E)$ $pred_s(E)$ | $d_s(F)$ $pred_s(F)$ |
|---|---|---|---|---|---|---|---|
| init | {A} | 0,– | 2,A | 5,A | 1,A | ∞ | ∞ |
| 1 | {A,D} | – | 2,A | **4,D** | – | **2,D** | ∞ |
| 2 | {A,D,E} | – | 2,A | **3,E** | – | – | **4,E** |
| 3 | {A,D,E,B} | – | – | 3,E | – | – | 4,E |
| 4 | {A,D,E,B,C} | – | – | – | – | – | 4,E |
| 5 | {A,D,E,B,C,F} | – | – | – | – | – | – |

WHAT IS THE NEXT HOP?

- For example, consider the shortest path from A to **F**. What is the **next hop** — the neighbour A should hand the packet to?
- The table gives the least cost, **4**, and F's predecessor **E** — the node visited just before F.
- Neither of those is the **next hop out of A**. So how do we find it?

DIJKSTRA'S ALGORITHM

# How do we build the forwarding table from this?

| Step | S | $d_s(A)$ $pred_s(A)$ | $d_s(B)$ $pred_s(B)$ | $d_s(C)$ $pred_s(C)$ | $d_s(D)$ $pred_s(D)$ | $d_s(E)$ $pred_s(E)$ | $d_s(F)$ $pred_s(F)$ |
|---|---|---|---|---|---|---|---|
| init | {A} | 0,– | 2,A | 5,A | **1,A** | ∞ | ∞ |
| 1 | {A,D} | – | 2,A | **4,D** | – | **2,D** | ∞ |
| 2 | {A,D,E} | – | 2,A | **3,E** | – | – | **4,E** |
| 3 | {A,D,E,B} | – | – | 3,E | – | – | 4,E |
| 4 | {A,D,E,B,C} | – | – | – | – | – | 4,E |
| 5 | {A,D,E,B,C,F} | – | – | – | – | – | – |

**TRACING F'S PREDECESSORS**

$pred_s(F) = E \rightarrow pred_s(E) = D \rightarrow pred_s(D) = A$

So the path is **A → D → E → F**. Therefore, on the shortest path to F, the **next hop is D**.

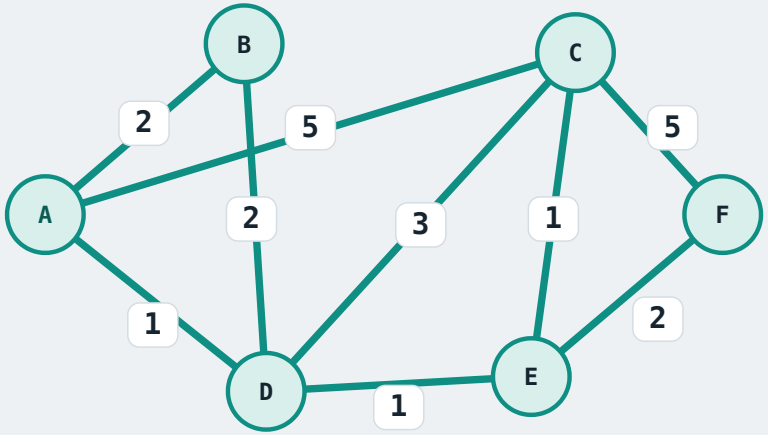
# How do we build the forwarding table from this?

| Step | S | $d_s(A)$ $pred_s(A)$ | $d_s(B)$ $pred_s(B)$ | $d_s(C)$ $pred_s(C)$ | $d_s(D)$ $pred_s(D)$ | $d_s(E)$ $pred_s(E)$ | $d_s(F)$ $pred_s(F)$ |
|---|---|---|---|---|---|---|---|
| init | {A} | 0,– | 2,A | 5,A | **1,A** | ∞ | ∞ |
| 1 | {A,D} | – | 2,A | **4,D** | – | **2,D** | ∞ |
| 2 | {A,D,E} | – | 2,A | **3,E** | – | – | **4,E** |
| 3 | {A,D,E,B} | – | – | 3,E | – | – | 4,E |
| 4 | {A,D,E,B,C} | – | – | – | – | – | 4,E |
| 5 | {A,D,E,B,C,F} | – | – | – | – | – | – |

Do this for every destination → A's forwarding table:

| **Dest** | B | C | D | E | F |
|---|---|---|---|---|---|
| **Next hop** | **B** | **D** | **D** | **D** | **D** |

# Why does Dijkstra need the full topology?




**1.** Each step picks the unsettled node with the **smallest** tentative distance — so the algorithm must know **which nodes exist** in the network.

**2.** When relaxing from a settled node u, the algorithm needs the edge weight c(u,w) for each of u's neighbours. Since every node is eventually settled, this means it needs **every edge weight** in the graph.

Together, Dijkstra needs every node, every edge and every edge cost — the **complete network topology**.

QUESTION

Dijkstra requires the **complete topology**. So how does each router obtain it?

LINK-STATE FLOODING

# How every router builds its map

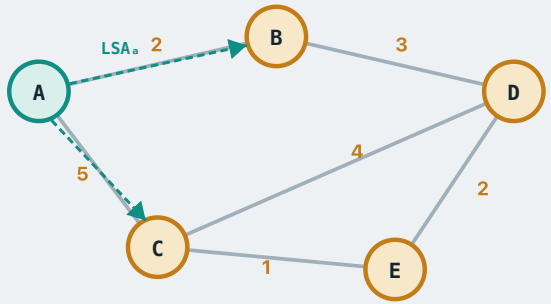
Each router starts with only what it knows locally: its **neighbours** and the cost of **its own links**. It packages this as a **Link-State Advertisement (LSA)**.

Each router **floods** its LSA — it sends the LSA to all its neighbours, who forward it to their neighbours, and so on until every other router has received it.

Each router stores all incoming LSAs in its **Link-State Database (LSDB)**. Once all LSAs have propagated, every router's LSDB is **identical** — they all share the same complete topology.

LINK-STATE FLOODING

# Building the global picture

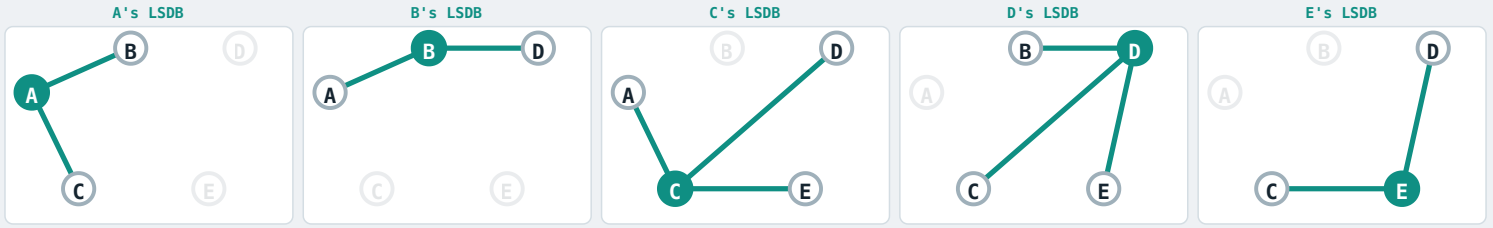



**Step 1**

Each router discovers its local neighbours and link costs. Below: the initial **Link-State Database (LSDB)** for every router.

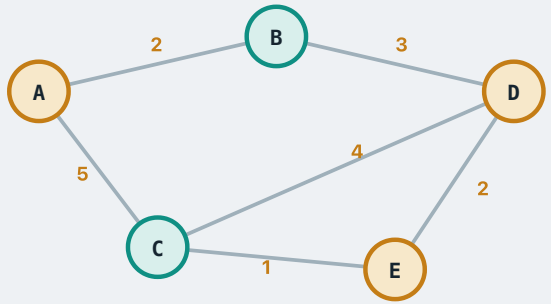
**Step 2**

A packages its local view into **LSA$_A$** and sends it to its neighbours B and C.



LINK-STATE FLOODING

# Building the global picture




**Step 1**

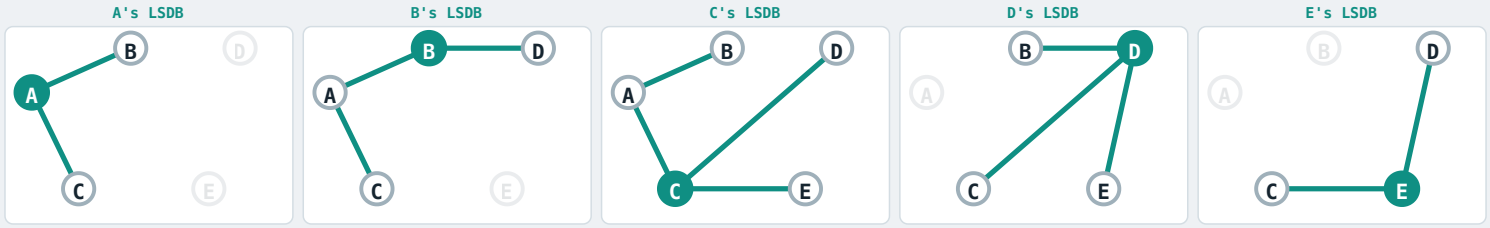
Each router discovers its local neighbours and link costs. Below: the initial **Link-State Database (LSDB)** for every router.

**Step 2**

A packages its local view into **$LSA_A$** and sends it to its neighbours B and C.

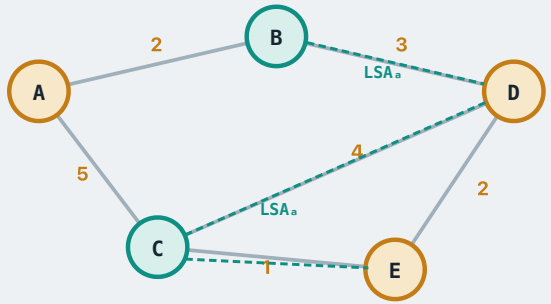
**Step 3**

B and C receive **$LSA_A$** and update their LSDB — their view now includes A's links.



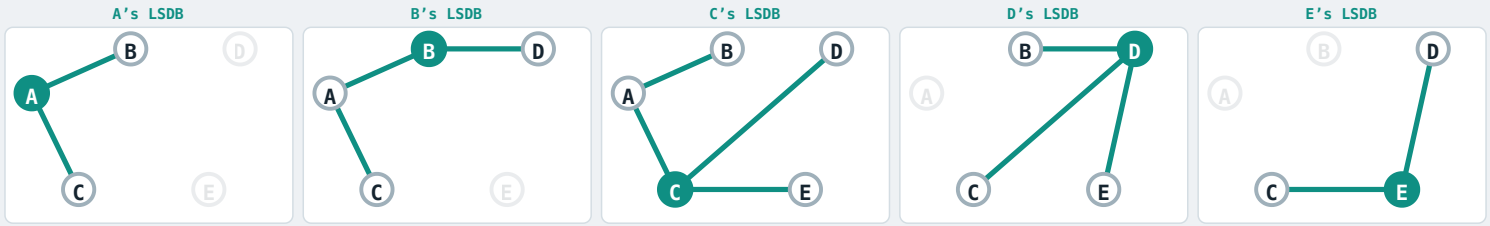
LINK-STATE FLOODING

# Building the global picture



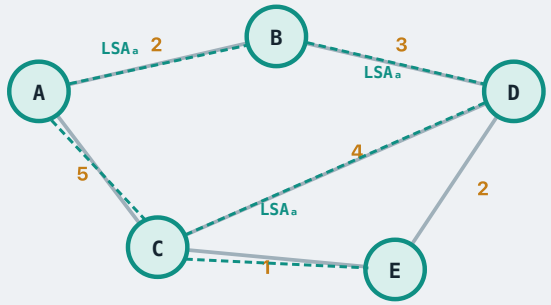

**Step 4**

B and C forward **$LSA_A$** onward: B → D, and C → D and E.



LINK-STATE FLOODING

# Building the global picture




**Step 4**

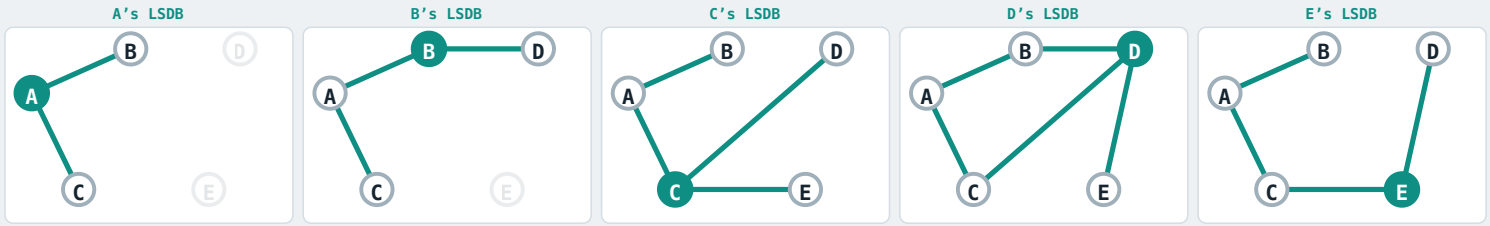
B and C forward **$LSA_A$** onward: B → D, and C → D and E.

**Step 5**

D and E receive **$LSA_A$** and update their LSDB.

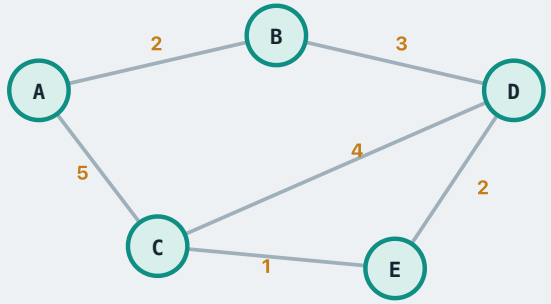
**Step 6**

This all happens in parallel: B, C, D, E simultaneously advertise their own LSAs.



LINK-STATE FLOODING

# Building the global picture




**Step 4**

B and C forward $\mathbf{LSA_A}$ onward: B → D, and C → D and E.

**Step 5**

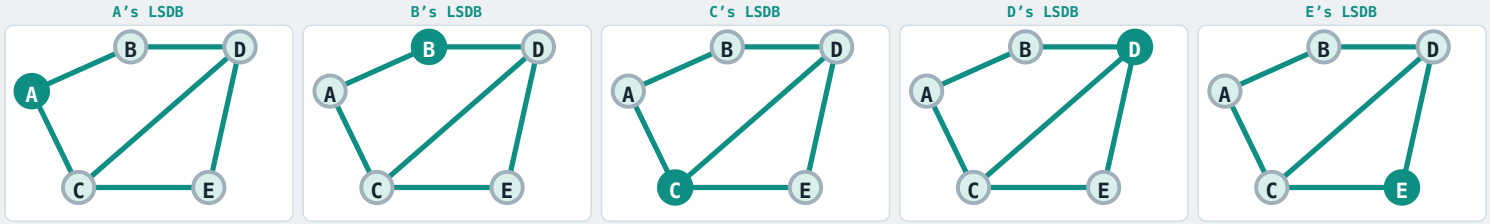
D and E receive $\mathbf{LSA_A}$ and update their LSDB.

**Step 6**

This all happens in parallel: B, C, D, E simultaneously advertise their own LSAs.

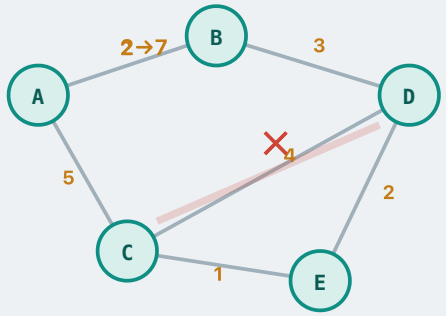
**Step 7**

After all LSAs propagate, every router's LSDB is **identical** — they all share the same complete topology.



WHEN THE MAP CHANGES

# Re-flooding




## Why do we need re-flooding?

**Topology change** — a link goes down, a router crashes, or a new link appears

**Cost change** — an administrator changes a link weight

**Periodic refresh** — every ~30 min, each router re-announces its links even if nothing has changed

**What happens** — the affected router generates a **fresh LSA** and re-floods it. Every router updates its LSDB.

THE TWO FAMILIES

# Distance-vector vs Link-state

| | DISTANCE-VECTOR | LINK-STATE |
|---|---|---|
| Approach | **Distributed** — only ever talks to its neighbours | **Centralised** — needs the full topology (LSDB) |
| Routing loops | **Prone** (count-to-infinity) | **Loop-free** (consistent topology) |
| Memory | Small (only distance vectors) | Large (full topology database / LSDB) |

ACKNOWLEDGEMENTS

# Lineage & colophon

These lecture notes modernise COSC264 material developed by **Dr Barry Wu** (2024–25), building on notes by **Dr Mengmeng Ge** (2023) and **Assoc Prof Dan Kim** (2017), University of Canterbury, and lecture notes by **Prof Aleksandar Kuzmanovic** (CS340, Northwestern University, 2005).